

PRACTICALS ON REAL ANALYSIS – II

$$\int_a^b f(x) dx$$
$$L(P, f) \leq \int_a^b f(x) dx \leq U(P, f)$$
$$\lim_{\|P\| \rightarrow 0} \sum_i f(\xi_i) \Delta x_i$$

WITH PYTHON



A Laboratory Manual for Undergraduate Mathematics

```
>>> real_analysis.py
```

REVISED NEP SYLLABUS
2026–27 Onwards

B.Sc. Mathematics, Semester V

Karnatak University, Dharwad

Course Code: C5MAT1P1

Dr. B Parvathalu

M.Sc., M.Tech., Ph.D.

Associate Professor

Department of Mathematics

Karnatak Arts/Science College, Dharwad

AUGUST 2026

B Parvathalu
Karnatak Arts/Science College, Dharwad
Karnataka, India

PRACTICALS ON REAL ANALYSIS–II WITH PYTHON **A Laboratory Manual for Undergraduate Mathematics**

Based on the Revised NEP Syllabus (2026–27)
B.Sc. Mathematics, Semester V
Karnatak University, Dharwad
Course Code: C5MAT1P1

Copyright © 2026 B Parvathalu

Prepared with the support of the Karnatak University Mathematics Teachers Association (KUMTA) during a Python Programming event conducted by Government First Grade College, Haliyal, Karnataka, India.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means without prior written permission of the copyright holder, except as permitted by law.

“Python” and the Python Logo are trademarks of the Python Software Foundation (PSF). This publication is not affiliated with or endorsed by the PSF.

First Edition: 2026
ISBN: 978-93-344-8833-3

Contents

Preface	4
0 Introduction and Basics of Python	5
0.1 Installation of Python and Python Development Environments	5
0.2 Integrated Development Environments	6
0.3 Basic Python and SymPy Syntax for Mathematical Computations	8
0.4 Naming Conventions in SymPy	16
0.5 Various Ways of Printing in Python	19
0.6 Common Errors When Writing Python Programs	23
1 Program to Find Upper and Lower Riemann Sums with Respect to a Given Partition	28
2 Program to Test Riemann Integrability	29
3 Program to Evaluate a Riemann Integral as a Limit of Sums	31
4 Program to Verify the First Mean Value Theorem of Integral Calculus	32
5 Program to Verify the Second Mean Value Theorem of Integral Calculus	35
6 Program to Verify the Fundamental Theorem of Calculus with Suitable Examples	37
7 Program to Verify the Convergence of an Improper Integral of the First Kind	39
8 Program to Verify the Convergence of an Improper Integral of the Second Kind	40
9 Program to Illustrate Dirichlet's Test and Abel's Test with Suitable Examples	41
10 Program to Evaluate Beta and Gamma Functions for Different Values	44
11 Program to Verify the Beta–Gamma Relation with Suitable Examples	46
12 Program to Verify the Recurrence Relation and Duplication Formula of the Gamma Function with Suitable Examples	47
Scheme of Examination	50
100 Quick Python Exercises Before Running the Main Programs	51
Bibliography	70

DEDICATED TO
THOSE WHO LEARN MATHEMATICS BY THINKING, EXPLORING AND COMPUTING.

Preface

Dear All,

This laboratory manual is prepared based on my experience and experimental learning in using modern computational technologies in Mathematics. It is my first attempt at preparing a manual of this kind and the entire document has been prepared using $\text{\LaTeX 2}_{\epsilon}$, a powerful typesetting system widely used for mathematical and scientific writing.

While preparing this manual, I have explored different ways of presenting mathematical computations through *Python*. However, I have tried to keep the programs simple and student friendly, without compromising mathematical understanding. Students are encouraged not merely to execute the programs but to try different examples, modify the codes and observe the results. Such experimentation will help them understand both the strengths and limitations of Python in mathematical computation. Before proceeding to the main programs, students should complete the relevant exercises in the section “100 Quick Python Exercises Before Running the Main Programs”.

I sincerely thank *Dr. Ranjana Bhandare* for inviting me to deliver a talk on Python at Government First Grade College, Haliyal. I am happy to say that this manual is largely an outcome of that opportunity. I also extend my heartfelt thanks to all the executive members of the *Karnatak University Mathematics Teachers Association (KUMTA)*, particularly *Shri. T. V. Marigoudar* and *Dr. Mahesh K Bhat*, for their encouragement and support to prepare this manual.

This manual is not intended to be a perfect or final work. As we know, nothing is perfect in this world; improvement depends on how we perceive and refine things. Therefore, constructive feedback is always welcome, especially suggestions for adding or removing programs, improving explanations, or including useful shortcuts that may benefit our students.

The source codes of the programs in this manual are also available on my personal GitHub web page: <https://drbp-kcd.github.io/>.

I hope this manual will serve as a useful source for students to explore Mathematics through Python with curiosity and confidence.

Happy Computing and Happy Learning !☺

Dr. B Parvathalu

0 Introduction and Basics of Python

Python is a high-level, interpreted and general purpose programming language. It was created by Guido van Rossum and first released in 1991. Python is popular because of its simple syntax, readability and wide range of applications.

Python is commonly used in web development, data analysis, artificial intelligence, machine learning, scientific computing and automation.

Features of Python

- Simple and easy to learn
- Free and open source
- Interpreted programming language
- Platform independent
- Large standard library
- Supports object-oriented programming

0.1 Installation of Python and Python Development Environments

To write and execute Python programs, the Python interpreter must first be installed on the computer. In addition to the interpreter, an Integrated Development Environment (IDE) or code editor can be used to write, execute, test and debug Python programs conveniently.

Installing Python

Python is free and open-source software and is available for major operating systems such as Windows, Linux and macOS. The official Python distribution can be obtained from the Python website:

<https://www.python.org/downloads/>

For Windows, the general installation procedure is as follows:

1. Download a current stable version of Python 3 from the official Python website.
2. Run the downloaded installer.
3. Select the option to add Python to the system **PATH**, when the installer provides this option.
4. Complete the installation.
5. Open Command Prompt or a terminal and verify the installation using:

```
python --version
```

On some systems, the command may be:

```
python3 --version
```

If Python is installed correctly, the installed Python version will be displayed.

Python programs are normally stored in files with the extension `.py`. For example,

```
hello.py
```

may contain:

```
print("Hello, Python!")
```

The program can be executed from a terminal using:

```
python hello.py
```

or, depending on the operating system,

```
python3 hello.py
```

Installing Python Packages

Python provides a package-management system called `pip`. It is commonly used to install additional Python libraries.

The availability of `pip` can be checked using:

```
python -m pip --version
```

A package can be installed using the general command:

```
python -m pip install package_name
```

For example, the SymPy library can be installed using:

```
python -m pip install sympy
```

After installation, SymPy can be imported into a Python program as:

```
from sympy import *
```

or

```
import sympy as sp
```

0.2 Integrated Development Environments

An **Integrated Development Environment (IDE)** provides tools for writing, executing, debugging and managing programs. Python can also be used with code editors and interactive environments that provide similar facilities. Some commonly used development environments are described below.

IDLE IDLE (Integrated Development and Learning Environment) is supplied with the standard Python distribution on many platforms. It provides a Python shell, source-code editor, syntax highlighting, automatic indentation and basic debugging facilities. It is simple to use and suitable for beginners.

Availability: Free

Thonny Thonny is a lightweight Python IDE designed primarily for beginners. It provides a simple editor, Python shell, step-by-step debugger, variable inspection and package installation facilities. It is well suited for introductory Python laboratory courses.

Availability: Free and open source

Visual Studio Code Visual Studio Code (VS Code) is a popular source-code editor developed by Microsoft. With the Python extension, it provides syntax highlighting, code completion, an integrated terminal, debugging, environment selection and source-control support. The Python interpreter must be installed separately.

Availability: Free

PyCharm PyCharm is a Python-focused IDE developed by JetBrains. It provides code completion, debugging, code inspection, refactoring and project-management tools. Core Python development features are available free of charge, while additional professional features require a subscription.

Availability: Free / Subscription

Spyder Spyder (Scientific Python Development Environment) is a free and open-source IDE designed particularly for scientific and numerical computing. It provides a source-code editor, interactive console, variable explorer, debugger and tools for inspecting plots and data.

Availability: Free and open source

JupyterLab JupyterLab is a free and open-source, web-based interactive development environment widely used for Python programming, scientific computing and data analysis. Programs can be divided into cells and executed individually, with results displayed immediately. It also supports mathematical expressions, plots, tables, files and terminals.

It can be installed and started using:

```
python -m pip install jupyterlab
jupyter lab
```

Availability: Free and open source

SageMath SageMath is a free and open-source mathematical software system that uses Python as its principal programming language. It supports symbolic computation, calculus, linear algebra, number theory, differential equations, numerical computation, graph theory and mathematical visualization. It can also be used through Jupyter notebooks.

Availability: Free and open source

Wing Python IDE Wing is a Python-specific IDE providing debugging, code navigation, testing, code intelligence and project-management facilities. It is intended particularly for larger and professional Python development projects.

Availability: Free / Paid

Comparison of Development Environments

Environment	Availability
IDLE	Free
Thonny	Free
VS Code	Free
PyCharm	Free / Subscription
Spyder	Free
JupyterLab	Free
SageMath	Free
Wing	Free / Paid

0.3 Basic Python and SymPy Syntax for Mathematical Computations

Before executing the programs, it is useful to become familiar with the basic Python and SymPy syntax commonly used in mathematical computations. Python uses indentation to define blocks of code, and the `print()` function is used to display results.

Comments in Python

Comments are used to explain the Python code and are ignored by the Python interpreter.

Single-Line Comment

A single-line comment begins with the hash symbol (`#`).

```
# This is a single-line comment
print("Hello, Python!")
```

Inline Comment

A comment can also be written after a Python statement.

```
x = 10  # Store the value 10 in x
```

Multi-Line Comment

Python does not have a separate syntax for multi-line comments. Multiple lines can be commented using the hash symbol (`#`).

```
# This is a multi-line comment.
# It spans several lines.
# Each line starts with #.
```

```
print("Hello, Python!")
```

Variables, Arithmetic and Lists. Python does not require explicit declaration of variable types. Variables may store numbers, strings, Boolean values, lists and other objects.

```
a = 10
b = 4
partition = [0, 1/4, 1/2, 3/4, 1]

print(a + b)      # Addition
print(a - b)      # Subtraction
print(a * b)      # Multiplication
print(a / b)      # Division
print(a**2)       # Exponentiation
```

The operators $+$, $-$, $*$, $/$ and $**$ represent addition, subtraction, multiplication, division and exponentiation, respectively. Thus, x^3 is written as `x**3`.

Importing SymPy. SymPy is a Python library for symbolic mathematics. In the programs of this manual, it is imported using

```
from sympy import *
```

After this command, SymPy functions such as `symbols()`, `simplify()`, `factor()`, `limit()`, `diff()`, `integrate()` and `solve()` can be used directly without a prefix.

For example,

```
from sympy import *

x = symbols('x')
f = x**2 + 2*x + 1

print(factor(f))
print(diff(f, x))
```

Symbolic Variables and Mathematical Expressions. Mathematical variables are created using `symbols()`.

```
x = symbols('x')
x, y = symbols('x y')
m, n = symbols('m n', positive=True)
r, n = symbols('r n', positive=True, integer=True)

f = x**2 + 3*x + 2
g = sin(x)
h = 1/(1 + x**2)
```

Assumptions such as `positive=True` and `integer=True` may be specified when required.

Common mathematical constants and functions include

```

pi          # pi
E           # e
oo          # infinity
sqrt(x)     # square root
sin(x), cos(x) # trigonometric functions
exp(x)      # exponential function
log(x)      # logarithmic function

```

For example,

```

f = sin(pi*x)
g = E**x
h = sqrt(x)

```

represent

$$f(x) = \sin(\pi x), \quad g(x) = e^x, \quad h(x) = \sqrt{x}.$$

Exact Fractions and Numerical Values. For exact symbolic calculations, fractions may be entered using `S()` or `Rational()`.

```

a = S(1)/2
b = S(3)/4
c = Rational(2, 3)

```

These represent exactly

$$\frac{1}{2}, \quad \frac{3}{4}, \quad \frac{2}{3}.$$

A numerical approximation of a symbolic expression can be obtained using `evalf()`.

```

print(sqrt(2))
print(sqrt(2).evalf())
print(pi.evalf())

```

Substitution and Algebraic Manipulation. The command `subs()` is used to substitute a value or expression for a symbolic variable.

```

f = x**2 + 1
print(f.subs(x, 2))

```

Here, `f.subs(x,2)` represents $f(2)$.

Some useful commands for algebraic manipulation are

```

simplify((x**2 - 1)/(x - 1))
expand((x + 1)**3)
factor(x**2 - 1)

```

Summation and Limits. A symbolic sum can be constructed using `Sum()`.

```
i, n = symbols('i n', positive=True, integer=True)

S1 = Sum(i, (i, 1, 5))
print(S1.doit())

S2 = Sum(i**2, (i, 1, n))
print(factor(S2.doit()))
```

The general form is

```
Sum(expression, (variable, lower_limit, upper_limit))
```

and `doit()` evaluates the sum whenever possible.

Limits are calculated using `limit()`.

```
limit(sin(x)/x, x, 0)
limit(1/x, x, oo)
limit((1 + S(1)/n)**n, n, oo)
```

For example,

$$\lim_{n \rightarrow \infty} \left(1 + \frac{1}{n}\right)^n = e.$$

One-sided limits may be calculated by

```
limit(f, x, a, dir='+')
limit(f, x, a, dir='-')
```

Differentiation and Integration. Symbolic differentiation is performed using `diff()`.

```
f = x**3 + 2*x**2 - x + 1

print(diff(f, x))      # First derivative
print(diff(f, x, 2))   # Second derivative
```

Indefinite and definite integrals are evaluated using `integrate()`.

```
f = x**2

F = integrate(f, x)
I = integrate(f, (x, 0, 1))

print(F)
print(I)
```

Thus,

$$\int x^2 dx \quad \text{and} \quad \int_0^1 x^2 dx$$

are computed symbolically.

Improper integrals can be represented using `oo`. For example,

```
integrate(1/(1+x**2), (x, 0, oo))
```

Solving Equations. The command `solve()` is used to solve equations symbolically.

```
x = symbols('x')
print(solve(x**2 - 4, x))
print(solve(Eq(x**2, 4), x))
```

Both commands solve the equation $x^2 = 4$.

Intervals, Minimum and Maximum. A closed interval $[a, b]$ can be represented using `Interval(a, b)`. The minimum and maximum values of a function on an interval may be obtained using `minimum()` and `maximum()`.

```
x = symbols('x')
f = x**2
I = Interval(0, 1)

m = minimum(f, x, I)
M = maximum(f, x, I)

print("Minimum =", m)
print("Maximum =", M)
```

These commands are particularly useful in computing lower and upper Riemann sums.

Beta and Gamma Functions. SymPy provides the special functions `beta()` and `gamma()`.

```
print(beta(2, 3))
print(gamma(4))
print(gamma(S(1)/2))
```

For symbolic parameters,

```
m, n = symbols('m n', positive=True)

B = beta(m, n)
G = gamma(n)
```

A decimal approximation may be obtained using

```
gamma(S(5)/3).evalf()
beta(S(3)/2, S(2)/3).evalf()
```

Conditional Statements and Verification. Conditional statements are useful for testing mathematical conditions.

```
a = 5

if a > 0:
    print("a is positive")
else:
    print("a is not positive")
```

Symbolic identities can often be verified by simplifying the difference between the two sides.

```
lhs = (x + 1)**2
rhs = x**2 + 2*x + 1

if simplify(lhs - rhs) == 0:
    print("Verified")
else:
    print("Not verified")
```

Indentation is essential in Python and determines the statements belonging to each block.

Loops and User-Defined Functions. A for loop is used when a calculation has to be repeated.

```
for i in range(1, 6):
    print(i)
```

For example, the subintervals of a partition can be accessed using

```
for i in range(len(partition) - 1):
    a = partition[i]
    b = partition[i + 1]
```

A reusable function is defined using the `def` keyword.

```
def square(x):
    return x**2

print(square(3))
```

A function involving symbolic integration may be written as

```
def integral(f, a, b):
    return integrate(f, (x, a, b))
```

Displaying Results. The `print()` function displays ordinary output, while `pprint()` provides a more readable mathematical form for symbolic expressions but *takes one positional argument*.

```
f = x**2
```

```
print("Function =", f)
print("Derivative =", diff(f, x))
print("Integral =", integrate(f, x))

pprint(integrate(f, x))
```

Special Values: ∞ , $-\infty$, ∞ and nan :

SymPy provides special objects to represent infinity, complex infinity and indeterminate numerical values. Some commonly used special values are ∞ , $-\infty$, ∞ and nan .

The required SymPy objects can be imported using:

```
from sympy import *
```

∞ — Positive Infinity The object ∞ represents positive infinity, denoted mathematically by $+\infty$. It is commonly used in limits, improper integrals and expressions that increase without bound.

For example,

```
from sympy import *

print( $\infty$  + 10)
print(1 /  $\infty$ )
```

The output is

```
 $\infty$ 
0
```

Thus,

$$\infty \longleftrightarrow +\infty.$$

$-\infty$ — Negative Infinity The object $-\infty$ represents negative infinity, denoted by $-\infty$.

For example,

```
from sympy import *

print( $-\infty$  - 10)
print(1 / ( $-\infty$ ))
```

The output is

```
 $-\infty$ 
0
```

Thus,

$$-\infty \longleftrightarrow -\infty.$$

zoo — Complex Infinity The object `zoo` represents *complex infinity*. It is used when the magnitude of a complex quantity becomes infinite without a specific real direction such as $+\infty$ or $-\infty$.

For example,

```
from sympy import *  
print(S(1) / 0)
```

The output is

```
zoo
```

Hence,

$$\text{zoo} \longleftrightarrow \text{complex infinity.}$$

Note that ordinary Python division by zero, such as `1/0`, raises a `ZeroDivisionError`. The use of `S(1)` converts the integer 1 into a SymPy object, so the operation is evaluated symbolically.

nan — Not a Number The object `nan` means *Not a Number*. It represents an indeterminate numerical result.

For example,

```
from sympy import *  
print(oo - oo)
```

The output is

```
nan
```

This corresponds to the indeterminate expression

$$\infty - \infty.$$

Thus, `nan` should not be interpreted as infinity. It indicates that a definite numerical value cannot be assigned to the expression.

is_finite Property

The `is_finite` property is used to determine whether a SymPy expression is known to be finite. It is used as a property and not as a function.

Its value may be:

- **True** — the expression is finite,
- **False** — the expression is infinite, or
- **None** — finiteness cannot be determined.

For example,

```
from sympy import *

print(Integer(5).is_finite)
print(oo.is_finite)
print((-oo).is_finite)
print(zoo.is_finite)
print(nan.is_finite)
```

The output is

```
True
False
False
False
None
```

Therefore, `oo`, `-oo` and `zoo` are not finite. For `nan`, SymPy does not assign either `True` or `False`; its `is_finite` value is `None`.

Comparison

Object	Meaning	is_finite
5	Finite number	True
oo	Positive infinity	False
-oo	Negative infinity	False
zoo	Complex infinity	False
nan	Not a Number	None

In summary,

```
oo → +∞,
-oo → -∞,
zoo → complex infinity,
nan → indeterminate numerical value.
```

The objects `oo`, `-oo`, `zoo` and `nan` represent special values, whereas `is_finite` is a property used to determine whether a SymPy expression is known to be finite.

0.4 Naming Conventions in SymPy

In SymPy, names beginning with a **lowercase letter** generally represent functions that perform mathematical operations, whereas names beginning with a **capital letter** generally represent classes used to construct mathematical objects.

For example,

`minimum()` is a function,

whereas

`Interval()` is a class.

Example: The `minimum()` Function

Consider the function

$$f(x) = x^2 - 4x + 5 = (x - 2)^2 + 1.$$

Hence, the minimum value of f is 1, attained at $x = 2$.

For the code:

```
from sympy import *

x = symbols('x')
f = x**2 - 4*x + 5
print(minimum(f, x))
```

The output is

```
1
```

Thus, `minimum()` is a function that performs the operation of finding a minimum value.

Example: The `Interval()` Class

An interval is a subset of the real numbers. SymPy provides the class `Interval()` to represent different types of intervals.

Type	Mathematical Form	SymPy Code
Closed	$[1, 5]$	<code>Interval(1, 5)</code>
Open	$(1, 5)$	<code>Interval.open(1, 5)</code>
Left-open	$(1, 5]$	<code>Interval.Lopen(1, 5)</code>
Right-open	$[1, 5)$	<code>Interval.Ropen(1, 5)</code>

For example, the open interval

$$(1, 5) = \{x \in \mathbb{R} : 1 < x < 5\}$$

is created using

For the code:

```
from sympy import *

A = Interval.open(1, 5)
print(A)
```

The output is

```
Interval.open(1, 5)
```

Similarly, the left-open interval

$$(1, 5] = \{x \in \mathbb{R} : 1 < x \leq 5\}$$

is created by

```
Interval.Lopen(1, 5)
```

and the right-open interval

$$[1, 5) = \{x \in \mathbb{R} : 1 \leq x < 5\}$$

is created by

```
Interval.Ropen(1, 5)
```

Thus, `Interval` starts with a capital letter because it is a class used to construct interval objects.

Example: `integrate()` and `Integral()`

Consider

$$\int x^2 dx.$$

The lowercase function `integrate()` performs the integration.

For the code:

```
from sympy import *
print(integrate(x**2, x))
```

The output is

$$x**3/3, \text{ that is, for } \frac{x^3}{3}.$$

On the other hand, the capitalized class `Integral()` creates a symbolic integral object.

For the code:

```
from sympy import *
I = Integral(x**2, x)
print(I)
```

It represents the unevaluated integral

$$\int x^2 dx.$$

It can be evaluated later using

```
I.doit()
```

which gives

$$x**3/3, \text{ that is, for } \frac{x^3}{3}.$$

Rule of Remember

lowercase name $\rightarrow$ usually a function or operation,
Capitalized name $\rightarrow$ usually a class or mathematical object.

For example,

`minimum()` $\rightarrow$ finds a minimum,
`integrate()` $\rightarrow$ performs integration,
`Interval()` $\rightarrow$ constructs an interval,
`Integral()` $\rightarrow$ constructs an integral object.

Note: This is a general Python/SymPy naming convention and not an absolute rule.

0.5 Various Ways of Printing in Python

Python provides several ways to display text, numbers, variables, and mathematical objects. The most commonly used method is the `print()` function.

Printing Text

A string can be printed by enclosing it in single or double quotes.

```
print("Hello, Python!")
```

Output:

```
Hello, Python!
```

Printing Variables

The value of a variable can be printed directly.

```
x = 10  
print(x)
```

Output:

```
10
```

Printing Text and Variables

Text and variables can be separated by commas in `print()`.

```
x = 10  
print("The value of x is", x)
```

Output:

```
The value of x is 10
```

Using f-Strings

An f-string allows variables or expressions to be inserted directly inside a string.

```
x = 10
y = 20
print(f"x = {x}, y = {y}")
```

Output:

```
x = 10, y = 20
```

Expressions can also be evaluated inside an f-string.

```
x = 5
print(f"The square of {x} is {x**2}")
```

Output:

```
The square of 5 is 25
```

Using the format() Method

The format() method is another way of inserting values into a string.

```
x = 10
y = 20
print("x = {}, y = {}".format(x, y))
```

Output:

```
x = 10, y = 20
```

Printing SymPy Objects

SymPy objects can be printed directly using print().

```
from sympy import *

x = symbols('x')
f = x**2 + 2*x + 1

print(f)
```

Output:

```
x**2 + 2*x + 1
```

Using pretty()

The SymPy function pretty() returns a string containing a two-dimensional mathematical representation of an expression.

```
s = pretty(f)
print(s)
```

The output is displayed approximately as

```
2
x  + 2.x + 1
```

Since `pretty()` returns a string, it is convenient when mathematical output has to be combined with ordinary text.

For example,

```
A = Interval.open(1, 5)
B = Interval.Lopen(1, 5)

print("Open interval:", pretty(A))
print("Left-open interval:", pretty(B))
```

Output:

```
Open interval: (1, 5)
Left-open interval: (1, 5]
```

Using `pprint()`

The SymPy function `pprint()` stands for *pretty print*. It displays a SymPy expression directly in a more readable mathematical form.

```
pprint(f)
```

The output is displayed approximately as

```
2
x  + 2*x + 1
```

It can also be used for sets and intervals. For example,

```
A = Interval.open(1, 5)
pprint(A)
```

Output:

```
(1, 5)
```

The main difference between `pretty()` and `pprint()` is that `pretty()` *returns* the formatted representation as a string, whereas `pprint()` *prints* the formatted representation directly.

<pre>pretty(expr) → returns a string, pprint(expr) → prints directly.</pre>

Hence, when some explanatory text and a SymPy object have to be printed together, `pretty()` with `print()` is often more convenient.

```
A = Interval.Ropen(1, 5)
print("Right-open interval:", pretty(A))
```

Output:

```
Right-open interval: [1, 5)
```

Controlling the Separator

By default, `print()` separates multiple objects by a space. The separator can be changed using the `sep` argument.

```
print(1, 2, 3, sep=", ")
```

Output:

```
1, 2, 3
```

Controlling the End Character

By default, `print()` ends with a new line. This behaviour can be changed using the `end` argument.

```
print("Hello", end=" ")
print("Python")
```

Output:

```
Hello Python
```

Printing Multiple SymPy Objects on the Same Line

Several strings and SymPy objects can be passed to `print()` by separating them with commas. Python inserts a space between the items automatically.

For example,

```
print("Open interval:", pretty(Interval.open(1, 5)),
      "and Left-open interval:", pretty(Interval.Lopen(1, 5)))
```

Output:

```
Open interval: (1, 5) and Left-open interval: (1, 5]
```

Similarly, all four types of intervals can be printed on the same line:

```
print("Closed:", pretty(Interval(1, 5)),
      "Open:", pretty(Interval.open(1, 5)),
      "Left-open:", pretty(Interval.Lopen(1, 5)),
      "Right-open:", pretty(Interval.Ropen(1, 5)))
```

Output:

```
Closed: [1, 5] Open: (1, 5) Left-open: (1, 5] Right-open: [1, 5)
```

Here, the comma-separated arguments of `print()` allow text and mathematical objects to be displayed together conveniently.

The same output can also be produced using an f-string:

```
A = Interval.open(1, 5)
B = Interval.Lopen(1, 5)

print(f"Open interval: {pretty(A)} and "
      f"Left-open interval: {pretty(B)}")
```

Output:

```
Open interval: (1, 5) and Left-open interval: (1, 5]
```

Thus, `print()`, f-strings, `format()`, `pretty()`, and `pprint()` provide convenient ways to display ordinary as well as mathematical objects in Python. For SymPy expressions, `pretty()` and `pprint()` are particularly useful for obtaining a more readable mathematical display.

The above syntax covers the principal Python and SymPy commands required for the mathematical programs in this laboratory manual. *Students are encouraged to execute these elementary examples first before proceeding to the main programs.*

0.6 Common Errors When Writing Python Programs

Errors are common while writing and executing Python programs. Reading the error message carefully usually helps to identify the problem. Some frequently encountered errors are given below.

Syntax Error. A syntax error occurs when a statement does not follow the rules of Python syntax. For example,

```
print("Hello"
```

produces an error because the closing parenthesis is missing. The correct statement is

```
print("Hello")
```

A colon must also be placed after statements such as `if`, `elif`, `else`, `for`, `while`, and `def`.

```
if x > 0:
    print("Positive")
```

Indentation Error. Python uses indentation to define a block of statements. Statements belonging to the same block must be indented consistently. Usually, four spaces are used for each level of indentation.

Incorrect:

```
if x > 0:
    print("Positive")
```

Correct:

```
if x > 0:
    print("Positive")
```

For nested statements, an additional four spaces are used at each level.

```
x = 10
if x > 0:
    if x % 2 == 0:
        print("Positive even number")
    else:
        print("Positive odd number")
else:
    print("Not positive")
```

Thus, *indentation is a part of Python syntax and must not be omitted.*

Undefined Variable or Name. A variable or symbolic object must be defined before it is used.

Incorrect:

```
print(x + 2)
```

Correct:

```
x = symbols('x')
print(x + 2)
```

Python is case-sensitive. Therefore, `x` and `X` are different names.

Using `^` Instead of `**`. In mathematical notation, x^2 denotes the square of x . In Python, exponentiation is written using `**`.

Incorrect:

```
f = x^2
```

Correct:

```
f = x**2
```

Similarly,

$$x^3 + 2x^2 + 1$$

is entered as

```
f = x**3 + 2*x**2 + 1
```

Missing Multiplication Symbol. Python requires the multiplication operator `*` to be written explicitly.

Incorrect:

```
f = 2x + 1
```

Correct:

```
f = 2*x + 1
```

Similarly, xy must be entered as `x*y`.

Incorrect Use of Equality. The symbol `=` is used for assignment, whereas `==` is used to test equality.

```
x = 5

if x == 5:
    print("Equal")
else:
    print("Not equal")
```

For symbolic equations, SymPy also provides `Eq()`.

```
equation = Eq(x**2, 4)
print(solve(equation, x))
```

Using SymPy Functions Without Importing SymPy. Commands such as `symbols()`, `diff()`, `integrate()`, and `limit()` require the SymPy library.

```
from sympy import *

x = symbols('x')
print(diff(x**2, x))
```

Exact Fractions and Floating-Point Numbers. For symbolic mathematical calculations, exact fractions are generally preferred.

Instead of

```
a = 0.5
```

one may use

```
a = S(1)/2
```

or

```
a = Rational(1, 2)
```

This allows SymPy to retain exact mathematical values.

Parentheses and Order of Operations. Parentheses must be placed carefully when entering mathematical expressions. For example,

$$\frac{1}{1+x^2}$$

should be entered as

```
f = 1/(1 + x**2)
```

and not as

```
f = 1/1 + x**2
```

because the two expressions have different meanings.

Errors in Lists and Indices. Python list indices begin with 0.

```
P = [0, S(1)/4, S(1)/2, S(3)/4, 1]

print(P[0])      # First element
print(P[1])      # Second element
```

For a list containing n elements, the valid indices are $0, 1, \dots, n-1$. Accessing an index outside this range produces an `IndexError`.

Errors in Loops. Statements belonging to a loop must be indented.

Incorrect:

```
for i in range(1, 6):
print(i)
```

Correct:

```
for i in range(1, 6):
    print(i)
```

For example, while working with the subintervals of a partition,

```
for i in range(len(partition) - 1):
    a = partition[i]
    b = partition[i + 1]
    delta = b - a
    print(a, b, delta)
```

all statements inside the loop are indented by four spaces.

Errors in User-Defined Functions. The body of a function must also be indented.

Incorrect:

```
def square(x):
    return x**2
```

Correct:

```
def square(x):
    return x**2
```

A mathematical function may contain several indented statements.

```
def definite_integral(f, a, b):
    I = integrate(f, (x, a, b))
    I = simplify(I)
    return I
```

Mathematical and Logical Errors. A program may execute without displaying an error message and still produce an incorrect mathematical result. Such errors are called logical errors.

For example,

```
integrate(f, (x, a, b))
```

and

```
integrate(f, (x, b, a))
```

are both syntactically correct, but they represent different mathematical quantities. Hence, formulas, limits, signs, brackets, assumptions, and conditions should always be checked carefully.

Reading Error Messages. Python normally displays the type of error and the location at which the error was detected. Some common errors are

Error	Possible Cause
SyntaxError	Incorrect Python syntax, missing colon, bracket, or quotation mark
IndentationError	Missing or inconsistent indentation
NameError	Use of an undefined variable or function
TypeError	Operation applied to an unsuitable type of object
ZeroDivisionError	Division by zero
IndexError	Attempt to access an invalid list index

When an error occurs, the indicated line and the immediately preceding line should be examined carefully. A missing parenthesis, quotation mark, or colon in an earlier line may cause an error to be reported on a later line.

1 Program to Find Upper and Lower Riemann Sums with Respect to a Given Partition

Definition 1.1 (Lower and Upper Riemann sums). For a bounded function $f : [a, b] \rightarrow \mathbb{R}$ and a partition $P : a = x_0 < x_1 < \dots < x_n = b$, define $m_r = \inf. f(x)$, $M_r = \sup. f(x)$ over the interval $I_r = [x_{r-1}, x_r]$ and $\Delta x_r = x_r - x_{r-1}$. The **lower Riemann sum** and **upper Riemann sum** are defined by

$$L(P, f) = \sum_{r=1}^n m_r \Delta x_r \text{ and } U(P, f) = \sum_{r=1}^n M_r \Delta x_r.$$

Experiment 1.1. Compute $L(P, f)$ and $U(P, f)$ for the function f defined by $f(x) = x^2$ on $[0, 1]$ and $P = \left\{0, \frac{1}{4}, \frac{2}{4}, \frac{3}{4}, 1\right\}$.

```
# 1. Program to find upper and lower Riemann sums
from sympy import *
x = symbols('x')
def riemann_sums(f, partition):
    lower_sum = S(0)
    upper_sum = S(0)
    for i in range(len(partition) - 1):
        a = partition[i]
        b = partition[i + 1]
        interval = Interval(a, b)
        m = minimum(f, x, interval)
        M = maximum(f, x, interval)
        delta = b - a
        lower_sum += m * delta
        upper_sum += M * delta
    return lower_sum, upper_sum
# ----- Calling for a Particular Problem -----
f = x**2
partition = [0, S(1)/4, S(2)/4, S(3)/4, 1]
L, U = riemann_sums(f, partition)
print("Lower Riemann Sum =", L)
print("Upper Riemann Sum =", U)
```

Lower Riemann Sum = 7/32
Upper Riemann Sum = 15/32

Practice Set 1.1.

1. Compute $L(P, f)$ and $U(P, f)$ for the function f defined by $f(x) = x$ on $[0, 1]$

and $P = \left\{0, \frac{1}{3}, \frac{2}{3}, 1\right\}$.

2. Compute $L(P, f)$ and $U(P, f)$ for the function f defined by $f(x) = x^3$ on $[0, 1]$ and $P = \left\{0, \frac{1}{5}, \frac{2}{5}, \frac{3}{5}, \frac{4}{5}, 1\right\}$.

3. Compute $L(P, f)$ and $U(P, f)$ for the function f defined by $f(x) = x^2$ on $[1, 2]$ and $P = \left\{1, \frac{6}{5}, \frac{7}{5}, \frac{8}{5}, \frac{9}{5}, 2\right\}$.

4. Compute $L(P, f)$ and $U(P, f)$ for the function f defined by $f(x) = e^x$ on $[0, 1]$ and $P = \left\{0, \frac{1}{3}, \frac{2}{3}, 1\right\}$.

2 Program to Test Riemann Integrability

Definition 2.1 (Lower and Upper Riemann Integrals). Let $f : [a, b] \rightarrow \mathbb{R}$ be a bounded function and let $\mathcal{P}$ denote the collection of all partitions P of $[a, b]$. The **lower Riemann integral** of f over $[a, b]$ is defined by

$$\int_a^b f(x) dx = \sup_{P \in \mathcal{P}} L(P, f)$$

and the **upper Riemann integral** of f over $[a, b]$ is defined by

$$\int_a^{\bar{b}} f(x) dx = \inf_{P \in \mathcal{P}} U(P, f).$$

Thus, the lower Riemann integral is the supremum of all lower sums, while the upper Riemann integral is the infimum of all upper sums.

Riemann Integral:

The function f is said to be **Riemann integrable** on $[a, b]$ if its lower and upper Riemann integrals are equal; that is,

$$\int_a^b f(x) dx = \int_a^{\bar{b}} f(x) dx.$$

The common value is called the **Riemann integral** of f over $[a, b]$ and is denoted by

$$\int_a^b f(x) dx.$$

As the number of subintervals increases and the norm of the partition tends to zero, the lower and upper Riemann sums approach the lower and upper Riemann integrals,

respectively. Thus, $\int_a^b f(x) dx = \lim_{n \rightarrow \infty} L(P, f)$ and $\int_a^{\bar{b}} f(x) dx = \lim_{n \rightarrow \infty} U(P, f)$,

where $\|P_n\| \rightarrow 0$ as $n \rightarrow \infty$.

Experiment 2.1. If f is defined on $[0, a]$; $a > 0$ by $f(x) = x^3 \forall x \in [0, a]$, then prove that f is Riemann integrable and $\int_0^a f(x) dx = \frac{a^4}{4}$.

```
# 2. Program to test Riemann Integrability
from sympy import *
x, r, n = symbols('x r n')
def riemann_integral(f, a, b):
    dx = (b - a)/n
    left = a + (r-1)*dx
    right = a + r*dx
    L = Sum(f.subs(x, left)*dx, (r, 1, n)).doit()
    U = Sum(f.subs(x, right)*dx, (r, 1, n)).doit()
    return limit(L, n, oo), limit(U, n, oo)
# ----- Calling for a Particular Problem -----
a = symbols('a')
f = x**3
LR, UR = riemann_integral(f, 0, a)
print("Lower Integral =", LR)
print("Upper Integral =", UR)
if LR == UR:
    print("The function is Riemann integrable.")
else:
    print("The function is not Riemann integrable.")
```

Lower Integral = $a^4/4$

Upper Integral = $a^4/4$

The function is Riemann integrable.

Practice Set 2.1.

1. If f is defined on $[0, a]$; $a > 0$ by $f(x) = x^2 \forall x \in [0, a]$, then prove that f is Riemann integrable and $\int_0^a f(x) dx = \frac{a^3}{3}$.
2. If f is defined on $[0, a]$; $a > 0$ by $f(x) = x \forall x \in [0, a]$, then prove that f is Riemann integrable and $\int_0^a f(x) dx = \frac{a^2}{2}$.
3. If f is defined on $[1, 2]$ by $f(x) = 2x + 1 \forall x \in [1, 2]$, then prove that f is Riemann integrable and $\int_1^2 f(x) dx = 4$.
4. If f is defined on $[1, 2]$ by $f(x) = 3x + 1 \forall x \in [1, 2]$, then prove that f is Riemann integrable and $\int_1^2 f(x) dx = \frac{11}{2}$.
5. If f is defined on $[a, b]$ by $f(x) = x \forall x \in [a, b]$, then prove that f is Riemann

$$\text{integrable and } \int_a^b f(x) dx = \frac{b^2 - a^2}{2}.$$

3 Program to Evaluate a Riemann Integral as a Limit of Sums

Definition 3.1 (Riemann integral as a limit of sum). Let $f : [a, b] \rightarrow \mathbb{R}$ be a bounded function. A partition P of $[a, b]$ is given by $P : a = x_0 < x_1 < \dots < x_n = b$, and let $\xi_r \in [x_{r-1}, x_r]$ be a point chosen in each subinterval. Define $\Delta x_r = x_r - x_{r-1}$.

The corresponding *Riemann sum* is $S(P, f) = \sum_{r=1}^n f(\xi_r) \Delta x_r$.

If there exists a number I such that $\lim_{\|P\| \rightarrow 0} \sum_{r=1}^n f(\xi_r) \Delta x_r = I$ for every choice of $\xi_r \in [x_{r-1}, x_r]$, where $\|P\| = \max_{1 \leq r \leq n} \Delta x_r$ is the norm (mesh) of the partition, then f is said to be *Riemann integrable* on $[a, b]$ and

$$\int_a^b f(x) dx = \lim_{\|P\| \rightarrow 0} \sum_{r=1}^n f(\xi_r) \Delta x_r = \lim_{n \rightarrow \infty} \sum_{r=1}^n f(\xi_r) \Delta x_r.$$

Experiment 3.1. Using the definition of the Riemann integral as the limit of Riemann sums, prove that

$$\int_0^1 (2x^2 - 3x + 5) dx = \frac{25}{6}.$$

```
# 3. Program to evaluate Riemann integral as a limit of sum
from sympy import *
x, r, n = symbols('x r n')
f = 2*x**2 - 3*x + 5
print("The function is continuous and hence integrable.")
a = 0
b = 1
dx = (b - a)/n
xr = a + r*dx
S = Sum(f.subs(x, xr)*dx, (r, 1, n)).doit()
print("Riemann Sum =", simplify(S))
# Limit of the Riemann sum
I = limit(S, n, oo)
print("Riemann integral as limit of sum =", simplify(I))
```

The function is continuous and hence integrable.

Riemann Sum = (25*n**2 - 3*n + 2)/(6*n**2)

Riemann integral as limit of sum = 25/6

Practice Set 3.1.

1. Using the definition of the Riemann integral as the limit of Riemann sums, prove that

$$\int_1^2 f(x) dx = 6, \text{ where } f(x) = 2x + 3.$$

2. Using the definition of the Riemann integral as the limit of Riemann sums, prove that

$$\int_1^2 x dx = \frac{3}{2}.$$

3. Using the definition of the Riemann integral as the limit of Riemann sums, evaluate

$$\int_1^3 (x^2 + 2x + 3) dx.$$

4. Using the definition of the Riemann integral as the limit of Riemann sums, evaluate

$$\int_0^2 (x - 2)^2 dx.$$

5. Using the definition of the Riemann integral as the limit of Riemann sums, evaluate

$$\int_0^2 (x^2 - 9x) dx.$$

4 Program to Verify the First Mean Value Theorem of Integral Calculus

Theorem 4.1 (First Mean Value Theorem for Integrals). *Let f and g be Riemann integrable on $[a, b]$, and suppose that g does not change sign on $[a, b]$. Then there exists a number μ , lying between the infimum and supremum of f on $[a, b]$, such that*

$$\int_a^b f(x)g(x) dx = \mu \int_a^b g(x) dx.$$

Corollary 4.1. *Let f be continuous on $[a, b]$ and g be Riemann integrable on $[a, b]$. If g does not change sign on $[a, b]$, then there exists $c \in [a, b]$ such that*

$$\int_a^b f(x)g(x) dx = f(c) \int_a^b g(x) dx.$$

Experiment 4.1. Verify the First Mean Value Theorem for the functions $f(x) = x^2$ and $g(x) = e^x$ on the interval $[1, 2]$.

```
# 4a. Program to verify the First Mean Value Theorem directly
from sympy import *
x = symbols('x')
f = x**2          # Define f(x)
g = exp(x)        # Define g(x)
a = 1             # Limits
b = 2
I1 = integrate(f * g, (x, a, b)) # Calculate the two integrals
I2 = integrate(g, (x, a, b))
mu = simplify(I1 / I2) # Find mu from the theorem
print("Integral of f(x)g(x) =", I1)
print("Integral of g(x) =", I2)
print("Value of mu =", mu)
infimum = minimum(f, x, Interval(a, b)) # Find infimum and supremum of f(x) on [a,b]
supremum = maximum(f, x, Interval(a, b))
print("Infimum of f(x) =", infimum)
print("Supremum of f(x) =", supremum)
LHS = I1 # Verify the theorem
RHS = mu * I2
print("Left hand side =", simplify(LHS))
print("Right hand side =", simplify(RHS))
if infimum <= mu <= supremum and simplify(LHS - RHS) == 0:
    print("First Mean Value Theorem is VERIFIED.")
else:
    print("First Mean Value Theorem is NOT verified.")
```

Integral of $f(x)g(x) = -E + 2\exp(2)$
 Integral of $g(x) = -E + \exp(2)$
 Value of $\mu = (1 - 2\exp(2))/(1 - \exp(2))$
 Infimum of $f(x) = 1$
 Supremum of $f(x) = 4$
 Left hand side = $-E + 2\exp(2)$
 Right hand side = $-E + 2\exp(2)$
 First Mean Value Theorem is VERIFIED.

Practice Set 4.1.

1. Verify the First Mean Value Theorem for the functions $f(x) = x$ and $g(x) = 1$ on the interval $[1, 2]$.
2. Verify the First Mean Value Theorem for the functions $f(x) = x^2$ and $g(x) = x$ on the interval $[0, 1]$.

3. Verify the First Mean Value Theorem for the functions $f(x) = x + 1$ and $g(x) = x$ on the interval $[1, 2]$.
4. Verify the First Mean Value Theorem for the functions $f(x) = x^2 + 1$ and $g(x) = 1$ on the interval $[0, 2]$.
5. Verify the First Mean Value Theorem for the functions $f(x) = 2x + 1$ and $g(x) = x^2$ on the interval $[0, 1]$.

Experiment 4.2. Using the First Mean Value Theorem, verify the following inequality:

$$\frac{1}{\pi} \leq \int_0^1 \frac{\sin \pi x}{1+x^2} dx \leq \frac{2}{\pi}.$$

```
# 4b. Program to verify the First Mean Value Theorem over inequalities
from sympy import *
x = symbols('x')
f = 1/(1+x**2) # f(x) and g(x)
g = sin(pi*x)
a = 0 # Limits
b = 1
I = integrate(f * g, (x, a, b)) # Calculate integral
G = integrate(g, (x, a, b)) # Calculate integral of g(x)
fc = I / G # First Mean Value Theorem: I = f(c) * G, Therefore f(c)
    ↪ = I/G
print("Integral I =", I)
print("Integral of g(x) =", G)
print("f(c) =", fc)
if 1/2 <= fc <= 1: # Check the bounds of f(c)
    print("First Mean Value Theorem verifies the required bounds.")
else:
    print("First Mean Value Theorem does not verify the required bounds.
    ↪")
```

```
Integral I = Integral(sin(pi*x)/(x**2 + 1), (x, 0, 1))
Integral of g(x) = 2/pi
f(c) = pi*Integral(sin(pi*x)/(x**2 + 1), (x, 0, 1))/2
First Mean Value Theorem verifies the required bounds.
```

Practice Set 4.2.

1. Using the First Mean Value Theorem, verify the following inequality:

$$\frac{\pi^2}{9} \leq \int_{\pi/6}^{\pi/2} \frac{x}{\sin x} dx \leq \frac{2\pi^2}{9}.$$

2. Using the First Mean Value Theorem, verify the following inequality:

$$\frac{1}{2} \leq \int_0^1 \frac{1}{1+x^2} dx \leq 1.$$

3. Using the First Mean Value Theorem, verify the following inequality:

$$\frac{1}{2} \leq \int_0^1 \frac{e^x}{1+x} dx \leq e - 1.$$

4. Using the First Mean Value Theorem, verify the following inequality:

$$\frac{2}{3} \leq \int_0^1 \frac{1+x}{1+x^2} dx \leq \frac{4}{3}.$$

5. Using the First Mean Value Theorem, verify the following inequality:

$$\frac{1}{3} \leq \int_0^1 \frac{x^2+1}{x+1} dx \leq \frac{2}{3}.$$

5 Program to Verify the Second Mean Value Theorem of Integral Calculus

Theorem 5.1 (Second Mean Value Theorem for Integrals – Weierstrass Form). *Let f be monotonic on $[a, b]$ and let g be Riemann integrable on $[a, b]$. Suppose that g does not change sign on $[a, b]$. Then there exists a point $c \in [a, b]$ such that*

$$\int_a^b f(x)g(x) dx = f(a) \int_a^c g(x) dx + f(b) \int_c^b g(x) dx.$$

Experiment 5.1. Verify the Second Mean Value Theorem for the functions $f(x) = x$ and $g(x) = e^x$ on the interval $[1, 2]$ by finding c value.

```
# 5a. Program to verify the Second Mean Value Theorem by finding c
↪value
from sympy import *
x,c = symbols('x c')
f = x # Define functions
g = E**x # or exp(x)
a = 1 # Limits
b = 2
I = integrate(f * g, (x, a, b)) # Original integral
# Find c from the Second Mean Value Theorem
```

```

c = solve(Eq(I, f.subs(x, a) * integrate(g, (x, a, c)) + f.subs(x, b) *
↳integrate(g, (x, c, b))), c)
print("Original integral =", I)
print("Value of c =", c)
# Verify with choice of c value
c_value = c[0]
RHS = f.subs(x, a) * integrate(g, (x, a, c_value)) + f.subs(x, b) *
↳integrate(g, (x, c_value, b))
print("Right hand side =", RHS)
print("Second Mean Value Theorem Verification:", simplify(I - RHS) == 0)

```

Original integral = $\exp(2)$

Value of c = $[\log(-E + \exp(2))]$

Right hand side = $\exp(2)$

Second Mean Value Theorem Verification: True

Practice Set 5.1.

1. Verify the Second Mean Value Theorem for the functions $f(x) = x$ and $g(x) = 1$ on the interval $[1, 2]$ by finding c value.
2. Verify the Second Mean Value Theorem for the functions $f(x) = x^2$ and $g(x) = 1$ on the interval $[0, 1]$ by finding c value.
3. Verify the Second Mean Value Theorem for the functions $f(x) = x + 1$ and $g(x) = x$ on the interval $[0, 1]$ by finding c value.
4. Verify the Second Mean Value Theorem for the functions $f(x) = 2x + 1$ and $g(x) = e^x$ on the interval $[0, 1]$ by finding c value.
5. Verify the Second Mean Value Theorem for the functions $f(x) = x^2 + 1$ and $g(x) = x + 1$ on the interval $[0, 1]$ by finding c value.

Experiment 5.2. Verify the Second Mean Value Theorem for the functions $f(x) = x^2$ and $g(x) = x$ on the interval $[1, 2]$ by taking $c = \frac{\sqrt{10}}{2}$.

```

# 5b. Program to verify the Second Mean Value Theorem with given
↳correct c value
from sympy import *
x = symbols('x')
f = x**2
g = x
a = 1
b = 2
c = sqrt(10)/2

```

```

LHS = integrate(f*g, (x, a, b))
RHS = f.subs(x,a)*integrate(g,(x,a,c)) + f.subs(x,b)*integrate(g,(x,c,b))
print("LHS =", LHS)
print("RHS =", RHS)
if LHS == RHS:
    print("Second Mean Value Theorem Verified")
else:
    print("Check the existence of correct c value")

```

LHS = 15/4

RHS = 15/4

Second Mean Value Theorem Verified

Practice Set 5.2.

1. Verify the Second Mean Value Theorem for the functions $f(x) = x$ and $g(x) = 1$ on the interval $[0, 2]$ by taking $c = 1$.
2. Verify the Second Mean Value Theorem for the functions $f(x) = x^2$ and $g(x) = 1$ on the interval $[0, 1]$ by taking $c = \frac{2}{3}$.
3. Verify the Second Mean Value Theorem for the functions $f(x) = x + 1$ and $g(x) = 1$ on the interval $[0, 1]$ by taking $c = \frac{1}{2}$.
4. Verify the Second Mean Value Theorem for the functions $f(x) = 2x + 1$ and $g(x) = 1$ on the interval $[0, 1]$ by taking $c = \frac{1}{2}$.
5. Verify the Second Mean Value Theorem for the functions $f(x) = 3x + 2$ and $g(x) = 1$ on the interval $[0, 2]$ by taking $c = 1$.

6 Program to Verify the Fundamental Theorem of Calculus with Suitable Examples

Theorem 6.1 (Fundamental Theorem of Integral Calculus). Let $f : [a, b] \rightarrow \mathbb{R}$ be Riemann integrable on $[a, b]$ and suppose that F is a primitive of f ; that is, $F'(x) = f(x)$ for all $x \in [a, b]$. Then

$$\int_a^b f(x) dx = F(b) - F(a).$$

Experiment 6.1. Verify the Fundamental Theorem of Calculus for the function $f(x) = x^3$ on the interval $[0, \frac{1}{2}]$.

```
# 6. Program to test Fundamental Theorem of Calculus Verification
from sympy import *
x = symbols('x')
f = x**3 # Function
F = integrate(f, x) # Find the indefinite integral
print("f(x) =", f)
print("F(x) = Primitive of f(x) =", F)
a = 0 # Definite integral limits
b = S(1)/2
integral = integrate(f, (x, a, b))
difference = F.subs(x, b) - F.subs(x, a)
print("Definite integral =", integral)
print("F(b) - F(a) =", difference)
# Verify FTC
print("Fundamental Theorem of Calculus Verification:", simplify(integral -
↪ difference) == 0)
```

```
f(x) = x**3
F(x) = Primitive of f(x) = x**4/4
Definite integral = 1/64
F(b) - F(a) = 1/64
Fundamental Theorem of Calculus Verification: True
```

Practice Set 6.1.

1. Verify the Fundamental Theorem of Calculus for the function $f(x) = x^2$ on the interval $[0, 1]$.
2. Verify the Fundamental Theorem of Calculus for the function $f(x) = 2x + 1$ on the interval $[1, 2]$.
3. Verify the Fundamental Theorem of Calculus for the function $f(x) = x^3 + 1$ on the interval $[0, 1]$.
4. Verify the Fundamental Theorem of Calculus for the function $f(x) = e^x$ on the interval $[0, 1]$.
5. Verify the Fundamental Theorem of Calculus for the function $f(x) = \sin x$ on the interval $[0, \frac{\pi}{2}]$.

7 Program to Verify the Convergence of an Improper Integral of the First Kind

Definition 7.1 (Improper Integral of the First Kind). The integral

$$\int_a^b f(x) dx$$

is said to be an **improper integral of the first kind** if the interval of integration is unbounded, that is, if at least one of the limits a or b is infinite, while f is bounded.

For example,

$$\int_1^{\infty} \frac{dx}{x^{\frac{3}{2}}}, \quad \int_{-\infty}^0 e^{3x} dx, \quad \int_{-\infty}^{\infty} \frac{dx}{x^2 + 3x + 4}$$

etc. are improper integrals of the first kind.

Experiment 7.1. Examine the convergence of the improper integral

$$\int_0^{\infty} \frac{dx}{1+x^2}.$$

```
# 7. Program to verify the convergence of an improper integral of
↳first kind.
from sympy import *
x, b = symbols('x b')
f = 1/(1+x**2)
I = integrate(f, (x, 0, b)) # Replace infinity by b and take the limit
L = limit(I, b, oo)
if L.is_finite:
    print("The improper integral is convergent and its value is", L)
else:
    print("The improper integral is divergent and its value is", L)
```

The improper integral is convergent and its value is $\pi/2$

Practice Set 7.1.

1. Examine the convergence of the improper integral

$$\int_1^{\infty} \frac{1}{x} dx.$$

2. Examine the convergence of the improper integral

$$\int_1^{\infty} \frac{dx}{\sqrt{x}}.$$

3. Examine the convergence of the improper integral

$$\int_1^{\infty} \frac{dx}{x^{3/2}}.$$

4. Examine the convergence of the improper integral

$$\int_0^{\infty} \sin x \, dx.$$

5. Examine the convergence of the improper integral

$$\int_0^{\infty} \frac{dx}{(1+x)^3}.$$

8 Program to Verify the Convergence of an Improper Integral of the Second Kind

Definition 8.1 (Improper Integral of the Second Kind). The integral

$$\int_a^b f(x) \, dx$$

is said to be an **improper integral of the second kind** if the interval of integration is bounded, but f is unbounded at one or more points of $[a, b]$.

For example,

$$\int_0^1 \frac{dx}{x^3}, \quad \int_1^3 \frac{dx}{3-x}, \quad \int_1^3 \frac{dx}{(x-1)(3-x)}$$

etc. are improper integrals of the second kind.

Experiment 8.1. Examine the convergence of the improper integral

$$\int_1^2 \frac{x}{\sqrt{x-1}} \, dx$$

```
# 8. Program to verify the convergence of an improper integral of
↪second kind.
from sympy import *
x, a = symbols('x a')
f = x / sqrt(x-1)
I = integrate(f, (x, a, 2)) # Replace the singular point by a and take
↪the right-hand limit
L = limit(I, a, 1, dir='+')
if L.is_finite:
    print("The improper integral is convergent and its value is", L)
else:
    print("The improper integral is divergent and its value is", L)
```

The improper integral is convergent and its value is $8/3$

Practice Set 8.1.

1. Examine the convergence of the improper integral

$$\int_0^1 \frac{dx}{\sqrt{x}}.$$

2. Examine the convergence of the improper integral

$$\int_0^1 \frac{dx}{x^2}.$$

3. Examine the convergence of the improper integral

$$\int_1^2 \frac{dx}{\sqrt{x-1}}.$$

4. Examine the convergence of the improper integral

$$\int_1^2 \frac{dx}{x\sqrt{x^2-1}}.$$

5. Examine the convergence of the improper integral

$$\int_2^3 \frac{x-1}{\sqrt{x-2}} dx.$$

9 Program to Illustrate Dirichlet's Test and Abel's Test with Suitable Examples

Theorem 9.1 (Dirichlet's Test). Suppose that $\int_a^t f(x) dx$ remains bounded for all $t \geq a$, and that g is a bounded monotonic function on $[a, \infty)$ satisfying $\lim_{x \rightarrow \infty} g(x) = 0$. Then the improper integral $\int_a^\infty f(x)g(x) dx$ converges.

Experiment 9.1. Examine the convergence of the integral $\int_1^\infty \frac{\sin x}{x} dx$ by Dirichlet's test.

```
# 9. Program to illustrate Dirichlet's test.
from sympy import *
```

```

x, t = symbols('x t', positive=True)
f = sin(x) # f(x) = sin(x)
g = 1/x # g(x) = 1/x
F = integrate(f, (x, 1, t)) # F(t) = integral from 1 to t of f(x)
if F not in [oo, -oo, zoo, nan]: # boundedness
    print("F(t) is bounded."); F_bounded = True
else:
    print("F(t) is not bounded."); F_bounded = False
g_limit = limit(g, x, oo) # Check boundedness using limit of g(x)
if g_limit not in [oo, -oo, zoo, nan]: # boundedness
    print("g(x) is bounded and its limit is", g_limit); g_bounded = True
else:
    print("g(x) is not bounded."); g_bounded = False
dg = diff(g, x) # Check monotonicity using derivative
if dg.is_negative or dg.is_positive:
    print("g(x) is monotonic."); g_monotonic = True
else:
    print("g(x) Monotonicity could not be determined."); g_monotonic = False
if F_bounded and g_bounded and g_monotonic and g_limit==0: # Dirichlet's test
    print("Hence, by Dirichlet's theorem, the given integral is convergent.")
else:
    print("Dirichlet's theorem cannot be applied.")

```

F(t) is bounded.

g(x) is bounded and its limit is 0

g(x) is monotonic.

Hence, by Dirichlet's theorem, the given integral is convergent.

Practice Set 9.1.

1. Examine the convergence of the integral $\int_1^{\infty} \frac{\sin x}{\sqrt{x}} dx$ by Dirichlet's test.
2. Examine the convergence of the integral $\int_1^{\infty} \frac{\sin x}{x^{3/2}} dx$ by Dirichlet's test.
3. Examine the convergence of the integral $\int_1^{\infty} \frac{\sin kx}{x} dx$ by Dirichlet's test.
4. Examine the convergence of the integral $\int_1^{\infty} \sin x^2 dx$ by Dirichlet's test

5. Examine the convergence of the integral $\int_1^{\infty} \frac{\cos x}{\sqrt{x+x^2}} dx$ by Dirichlet's test

Theorem 9.2 (Abel's Test). Suppose that $\int_a^{\infty} f(x) dx$ converges and that g is a bounded monotonic function on $[a, \infty)$. Then the improper integral $\int_a^{\infty} f(x)g(x) dx$ also converges.

Experiment 9.2. Examine the convergence of the integral $\int_1^{\infty} e^{-x} \frac{\sin x}{x^2} dx$ by Abel's test.

```
# 9b. Program to illustrate Abel's test.
from sympy import *
x, t = symbols('x t', positive=True)
f = sin(x)/x**2 # f(x) = sin(x)
g = E**(-x) # g(x) = 1/x
F = integrate(f, (x, 1, oo)) # F(t) = integral from 1 to t of f(x)
if F not in [oo, -oo, zoo, nan]: # F convergent
    print("Integral over f(x) is convergent."); F_convergent = True
else:
    print("Integral over f(x) is not convergent."); F_convergent = False
g_bound = limit(g, x, oo) # Check boundedness using limit of g(x)
if g_bound not in [oo, -oo, zoo, nan]: # boundedness
    print("g(x) is bounded."); g_bounded = True
else:
    print("g(x) is not bounded."); g_bounded = False
dg = diff(g, x) # Check monotonicity using derivative
if dg.is_negative or dg.is_positive:
    print("g(x) is monotonic."); g_monotonic = True
else:
    print("g(x) Monotonicity could not be determined."); g_monotonic = False
if F_convergent and g_bounded and g_monotonic: # Dirichlet's test
    print("Hence, by Abel's theorem, the given integral is convergent.")
else:
    print("Abel's theorem cannot be applied.")
```

Integral over f(x) is convergent.

g(x) is bounded.

g(x) is monotonic.

Hence, by Abel's theorem, the given integral is convergent.

Practice Set 9.2.

1. Examine the convergence of the integral $\int_1^{\infty} e^{-x} \frac{\sin x}{x} dx$ by Abel's test.
2. Examine the convergence of the integral $\int_1^{\infty} e^{-x} \frac{\sin x}{x^3} dx$ by Abel's test.
3. Examine the convergence of the integral $\int_1^{\infty} (1 - e^{-x}) \frac{\cos x}{x^2} dx$ by Abel's test.
4. Examine the convergence of the integral $\int_1^{\infty} e^{-x} \frac{\sin x}{x^4} dx$ by Abel's test.
5. Examine the convergence of the integral $\int_1^{\infty} (1 - e^{-x}) \frac{\cos x}{x^3} dx$ by Abel's test.

10 Program to Evaluate Beta and Gamma Functions for Different Values

Definition 10.1 (Beta Function). For $m > 0$ and $n > 0$, the integral

$$\int_0^1 x^{m-1}(1-x)^{n-1} dx$$

defines a function of m and n , called the **Beta function**, and is denoted by $B(m, n)$. Thus,

$$B(m, n) = \int_0^1 x^{m-1}(1-x)^{n-1} dx, \quad m > 0, \quad n > 0.$$

The Beta function is also known as the **First Eulerian Integral**.

For example,

$$1. \int_0^1 x^2(1-x)^4 dx = B(2+1, 4+1) = B(3, 5).$$

$$2. \int_0^1 x^{3/2}(1-x)^2 dx = B\left(\frac{3}{2}+1, 2+1\right) = B\left(\frac{5}{2}, 3\right).$$

$$3. \int_0^1 x^{-1/4}(1-x)^{-2/3} dx = B\left(-\frac{1}{4}+1, -\frac{2}{3}+1\right) = B\left(\frac{3}{4}, \frac{1}{3}\right).$$

4. $\int_0^1 x^{-4}(1-x)^5 dx$ is not a Beta function since $m = -4 + 1 = -3 < 0$.

Definition 10.2 (Gamma Function). For $n > 0$, the integral

$$\int_0^{\infty} x^{n-1} e^{-x} dx$$

defines a function of n , called the **Gamma function** and is denoted by $\Gamma(n)$. Thus,

$$\Gamma(n) = \int_0^{\infty} x^{n-1} e^{-x} dx, \quad n > 0.$$

The Gamma function is also known as the **Second Eulerian Integral**.

For example,

1. $\int_0^{\infty} x^4 e^{-x} dx = \Gamma(4 + 1) = \Gamma(5)$.
2. $\int_0^{\infty} x^{3/2} e^{-x} dx = \Gamma\left(\frac{3}{2} + 1\right) = \Gamma\left(\frac{5}{2}\right)$.
3. $\int_0^{\infty} x^{-1/2} e^{-x} dx = \Gamma\left(-\frac{1}{2} + 1\right) = \Gamma\left(\frac{1}{2}\right)$.

Experiment 10.1. Evaluate:

1. $B(4, 6)$, $B\left(\frac{3}{2}, 4\right)$, $B\left(\frac{3}{2}, \frac{1}{2}\right)$ and $B\left(\frac{3}{2}, \frac{2}{3}\right)$.
2. $\Gamma(4)$, $\Gamma\left(\frac{1}{2}\right)$, $\Gamma\left(\frac{7}{2}\right)$ and $\Gamma\left(\frac{5}{3}\right)$.

```
# 10. Program to evaluate Beta and Gamma functions for different
↪ values.
from sympy import *
# Evaluate Beta function
print("Beta(4, 6) =", beta(4, 6).rewrite(gamma))
print("Beta(3/2, 4) = ", beta(S(3)/2, 4).rewrite(gamma))
print("Beta(3/2, 1/2) =", beta(S(3)/2, S(1)/2).rewrite(gamma))
print("Beta(3/2, 2/3) =", beta(S(3)/2, S(2)/3).evalf())
# Evaluate Gamma function
print("Gamma(4) =", gamma(4))
print("Gamma(1/2) =", gamma(S(1)/2))
print("Gamma(7/2) =", gamma(S(7)/2))
```

```
print("Gamma(5/3) =", gamma(S(5)/3).evalf())
```

```
Beta(4, 6) = 1/504
Beta(3/2, 4) = 32/315
Beta(3/2, 1/2) = pi/2
Beta(3/2, 2/3) = 1.10876123966991
Gamma(4) = 6
Gamma(1/2) = sqrt(pi)
Gamma(7/2) = 15*sqrt(pi)/8
Gamma(5/3) = 0.902745292950934
```

Practice Set 10.1.

1. Evaluate
 - (a) $B(4, 6)$, $B\left(\frac{1}{2}, \frac{1}{2}\right)$, $B\left(\frac{3}{2}, \frac{1}{2}\right)$ and $B\left(\frac{1}{3}, \frac{2}{3}\right)$.
 - (b) $\Gamma(4)$, $\Gamma\left(\frac{1}{2}\right)$, $\Gamma\left(\frac{3}{2}\right)$ and $\Gamma\left(\frac{5}{3}\right)$.
2. Evaluate
 - (a) $B(3, 5)$, $B\left(\frac{1}{2}, \frac{3}{2}\right)$, $B\left(\frac{1}{4}, \frac{3}{4}\right)$ and $B\left(\frac{2}{3}, \frac{1}{3}\right)$.
 - (b) $\Gamma(5)$, $\Gamma\left(\frac{3}{2}\right)$, $\Gamma\left(\frac{5}{2}\right)$ and $\Gamma\left(\frac{4}{3}\right)$.
3. Evaluate
 - (a) $B(2, 7)$, $B\left(\frac{3}{2}, \frac{3}{2}\right)$, $B\left(\frac{1}{3}, \frac{2}{3}\right)$ and $B\left(\frac{1}{4}, \frac{1}{2}\right)$.
 - (b) $\Gamma(6)$, $\Gamma\left(\frac{1}{2}\right)$, $\Gamma\left(\frac{7}{2}\right)$ and $\Gamma\left(\frac{2}{3}\right)$.
4. Evaluate
 - (a) $B(5, 3)$, $B\left(\frac{1}{2}, 2\right)$, $B\left(\frac{3}{2}, 2\right)$ and $B\left(\frac{2}{3}, \frac{4}{3}\right)$.
 - (b) $\Gamma(3)$, $\Gamma\left(\frac{3}{2}\right)$, $\Gamma\left(\frac{9}{2}\right)$ and $\Gamma\left(\frac{5}{3}\right)$.
5. Evaluate
 - (a) $B(6, 4)$, $B\left(\frac{1}{2}, 3\right)$, $B\left(\frac{5}{2}, \frac{1}{2}\right)$ and $B\left(\frac{3}{4}, \frac{1}{4}\right)$.
 - (b) $\Gamma(7)$, $\Gamma\left(\frac{5}{2}\right)$, $\Gamma\left(\frac{7}{2}\right)$ and $\Gamma\left(\frac{7}{3}\right)$.

11 Program to Verify the Beta–Gamma Relation with Suitable Examples

Relation Between Beta and Gamma Functions:

$$B(m, n) = \frac{\Gamma(m)\Gamma(n)}{\Gamma(m+n)}, \quad \text{where } m > 0, n > 0.$$

Experiment 11.1. Verify the relation between Beta and Gamma functions:

$$B(m, n) = \frac{\Gamma(m)\Gamma(n)}{\Gamma(m+n)}, \quad \text{where } m > 0, n > 0.$$

```
# 11. Program to verify the Beta-Gamma relation.
from sympy import *
m, n = symbols('m n', positive=True)
lhs = beta(m, n)
rhs = gamma(m) * gamma(n) / gamma(m + n)
print("Beta(m,n) is given by: ", lhs)
print("Gamma relation is given by: ", rhs)
print("Verification of Beta-Gamma Relation:", simplify(lhs - rhs) == 0)
```

Beta(m,n) is given by: beta(m, n)

Gamma relation is given by: gamma(m)*gamma(n)/gamma(m + n)

Verification of Beta-Gamma Relation: True

Practice Set 11.1.

1. Prove that $\Gamma\left(\frac{1}{2}\right) = \sqrt{\pi}$.
2. Prove that $B\left(\frac{1}{n}, \frac{1}{2}\right) = \frac{\sqrt{\pi} \Gamma\left(\frac{1}{n}\right)}{\Gamma\left(\frac{1}{n} + \frac{1}{2}\right)}$.
3. Prove that $\int_0^\infty e^{-x^2} dx = \frac{\sqrt{\pi}}{2}$.
4. Prove that $\int_{-\infty}^0 e^{-x^2} dx = \frac{\sqrt{\pi}}{2}$.
5. Prove that $\int_{-\infty}^\infty e^{-x^2} dx = \sqrt{\pi}$.

12 Program to Verify the Recurrence Relation and Duplication Formula of the Gamma Function with Suitable Examples

Recurrence Formula for Gamma Function:

$$\Gamma(n+1) = (n)\Gamma(n), \quad \text{where } n > 0.$$

Duplication Formula for Gamma Function:

$$\Gamma(m)\Gamma\left(m + \frac{1}{2}\right) = \frac{\sqrt{\pi}}{2^{2m-1}}\Gamma(2m), \text{ where } m > 0.$$

Experiment 12.1. Verify the Recurrence and Duplication Formulas:

$$\Gamma(n+1) = (n)\Gamma(n) \text{ and } \Gamma(m)\Gamma\left(m + \frac{1}{2}\right) = \frac{\sqrt{\pi}}{2^{2m-1}}\Gamma(2m), \text{ where } m > 0 \text{ and } n > 0.$$

```
# 12. Program to verify the recurrence relation and duplication
↪ formula of the Gamma function
from sympy import *
n, m = symbols('n m', positive=True)
# To verify the recurrence relation
lhs = gamma(n + 1)
rhs = n * gamma(n)
print("LHS =", lhs)
print("RHS =", rhs)
print("Recurrence Relation Verification:", simplify(lhs - rhs) == 0)
# To verify the duplication formula
lhs = gamma(m) * gamma(m + S(1)/2)
rhs = 2**((1 - 2*m) * sqrt(pi)) * gamma(2*m)
print("LHS =", simplify(lhs))
print("RHS =", simplify(rhs))
print("Duplication Formula Verification:", simplify(lhs - rhs) == 0)
```

```
LHS = gamma(n + 1)
RHS = n*gamma(n)
Recurrence Relation Verification: True
LHS = gamma(m)*gamma(m + 1/2)
RHS = 2**((1 - 2*m)*sqrt(pi))*gamma(2*m)
Duplication Formula Verification: True
```

Practice Set 12.1.

1. Prove that $\Gamma\left(n + \frac{1}{2}\right) = \frac{\sqrt{\pi}\Gamma(2n+1)}{2^{2n}\Gamma(n+1)}$.
2. Prove that $B(m, m) \cdot B\left(m + \frac{1}{2}, m + \frac{1}{2}\right) = \frac{\pi m^{-1}}{2^{4m-1}}$.
3. Prove that $B(m, m) = 2^{1-2m}B\left(m, \frac{1}{2}\right)$.
4. Prove that $\Gamma\left(\frac{1}{4}\right)\Gamma\left(\frac{3}{4}\right) = \sqrt{2}\pi$.

5. If n is a positive integer, then prove that $\Gamma(n + 1) = n!$.

Scheme of Examination

Discipline Specific Core Course (DSC)

Course Title: Practical on Real Analysis – II

Course Code: C5MAT1P1 (Practical)

Compulsory Practical

Type of Course	Theory / Practical	Credits	Instruction hours per week	Total No. of Lectures / Hours per Semester	Duration of Sem End Exam	Formative Assessment Marks	Summative Assessment Marks	Total Marks
DSC – 5 (P5)	Practical	02	04	56 Hrs.	03 Hrs.	10	40	50

Distribution of Marks

Formative Assessment	
Assessment Occasion / Type	Marks
Program writing and problem solving (1 Program)	03
Program Execution	03
Viva	02
Journal	02
Total	10 Marks
<i>Formative Assessment as per guidelines.</i>	

Summative Assessment	
Assessment Occasion / Type	Marks
Program writing and problem solving (2 Programs)	$(5 + 5) \times 2 = 20$
Program Execution (Any 1 Program)	10
Viva	05
Journal	05
Total	40 Marks
<i>Summative Assessment as per guidelines.</i>	

100 Quick Python Exercises Before Running the Main Programs

Note: For each of the following exercises, add this as the first line:

```
from sympy import *
```

1. Given the partition $P = \left\{0, \frac{1}{4}, \frac{2}{4}, \frac{3}{4}, 1\right\}$, write a Python program using SymPy to display the partition exactly as given, without converting its elements into decimal numbers.

```
# Store the partition as an array with exact fractions.
P = [0, S(1)/4, S(2)/4, S(3)/4, 1]
# Display the partition using braces.
print(FiniteSet(*P))
```

```
{0, 1/4, 1/2, 3/4, 1}
```

2. Write a Python program to find the first and last elements of the partition $P = \left\{0, \frac{1}{4}, \frac{2}{4}, \frac{3}{4}, 1\right\}$.

```
# Store the partition as an array with exact fractions.
P = [0, S(1)/4, S(2)/4, S(3)/4, 1]
# Display the the first and last elements
print("First element:", P[0])
print("Last element:", P[-1])
```

```
First element: 0
```

```
Last element: 1
```

3. Write a Python program to find the number of partition points and the number of sub-intervals determined by the partition $P = \left\{0, \frac{1}{4}, \frac{2}{4}, \frac{3}{4}, 1\right\}$.

```
# Store the partition as an array with exact fractions.
P = [0, S(1)/4, S(2)/4, S(3)/4, 1]
# Display the the the number of partition points and the number of
↪subintervals
print("Number of partition points:", len(P))
print("Number of subintervals:", len(P)-1)
```

```
Number of partition points: 5
```

```
Number of subintervals: 4
```

4. Write a Python program to display all the sub-intervals determined by the partition $P = \left\{0, \frac{1}{4}, \frac{2}{4}, \frac{3}{4}, 1\right\}$.

```
# Store the partition as an array with exact fractions.
P = [0, S(1)/4, S(2)/4, S(3)/4, 1]
# Display all the subintervals by using pretty()
for i in range(len(P) - 1):
    print(pretty(Interval(P[i], P[i + 1])))
```

```
[0, 1/4]
[1/4, 1/2]
[1/2, 3/4]
[3/4, 1]
```

5. Write a Python program to find the length of each sub-interval determined by the partition $P = \left\{0, \frac{1}{4}, \frac{2}{4}, \frac{3}{4}, 1\right\}$.

```
# Store the partition as an array with exact fractions.
P = [0, S(1)/4, S(2)/4, S(3)/4, 1]
for i in range(len(P) - 1):# Display Length of each subinterval
    print("Length of subinterval", i + 1, "is", P[i + 1] - P[i])
```

```
Length of subinterval 1 is 1/4
Length of subinterval 2 is 1/4
Length of subinterval 3 is 1/4
Length of subinterval 4 is 1/4
```

6. Write a Python program to find the norm $\|P\|$ of the partition $P = \left\{0, \frac{1}{5}, \frac{2}{4}, \frac{3}{4}, 1\right\}$, where $\|P\| = \max_{1 \leq i \leq n} (x_i - x_{i-1})$.

```
# Store the partition as an array with exact fractions.
P = [0, S(1)/5, S(2)/4, S(3)/4, 1]
lengths = [P[i + 1] - P[i] for i in range(len(P) - 1)]
# Display Norm of the partition
print("Norm of the partition:", max(lengths))
```

```
Norm of the partition: 3/10
```

7. For the function $f(x) = x^2$, write a Python program using SymPy to find the infimum and supremum of f on the closed interval $\left[\frac{1}{2}, 1\right]$.

```
x = symbols('x', real=True)
f = x**2
infimum = minimum(f, x, Interval(S(1)/2, 1))
supremum = maximum(f, x, Interval(S(1)/2, 1))
print("Infimum:", infimum)
print("Supremum:", supremum)
```

```
Infimum: 1/4
Supremum: 1
```

8. For $f(x) = x^2$ and the partition $P = \left\{0, \frac{1}{4}, \frac{2}{4}, \frac{3}{4}, 1\right\}$, write a Python program to find the infimum and supremum of f on each sub-interval determined by P .

```
x = symbols('x', real=True)
f = x**2
P = [0, S(1)/4, S(2)/4, S(3)/4, 1]
for i in range(len(P) - 1):
    infimum = minimum(f, x, Interval(P[i], P[i + 1]))
    supremum = maximum(f, x, Interval(P[i], P[i + 1]))
    print("Over the Interval", [P[i], P[i + 1]], "Infimum =",
    infimum, " and Supremum =", supremum)
```

Over the Interval [0, 1/4] Infimum = 0 and Supremum = 1/16
 Over the Interval [1/4, 1/2] Infimum = 1/16 and Supremum = 1/4
 Over the Interval [1/2, 3/4] Infimum = 1/4 and Supremum = 9/16
 Over the Interval [3/4, 1] Infimum = 9/16 and Supremum = 1

9. write a Python program to find the set of all Real and Complex solutions of the equation $(x^2 + 1)(x^2 - 1) = 0$.

```
real_sol=solveset((x**2 - 1)*(x**2 + 1), x, domain=S.Reals)
complex_sol=solveset((x**2 - 1)*(x**2 + 1), x, domain=S.Complexes)
print("The set of Real solutions", real_sol)
print("The set of Complex solutions", complex_sol)
```

The set of Real solutions {-1, 1}
 The set of Complex solutions {-1, 1, -I, I}

10. For the function $f(x) = x + x^2$, write a Python program using SymPy to find the infimum and supremum of f on the semi-closed interval $[0, 1)$. Also determine whether the infimum and supremum are attained by the function on the given interval.

```
x = symbols('x', real=True)
f = x + x**2
interval = Interval(0, 1, right_open=True)
infimum = minimum(f, x, interval)
supremum = maximum(f, x, interval)
# Find points in the interval where each value is attained.
inf_points = solveset(f - infimum, x, domain=interval)
sup_points = solveset(f - supremum, x, domain=interval)
print("Infimum:", infimum)
print("Supremum:", supremum)
print("Infimum attained at:", inf_points)
print("Supremum attained at:", sup_points)
```

Infimum: 0
 Supremum: 2
 Infimum attained at: {0}
 Supremum attained at: EmptySet

11. For the function $f(x) = x + x^2$, write a Python program using SymPy to find the infimum and supremum of f on the semi-closed interval $(2, 3]$. Also determine whether the infimum and supremum are attained by the function on the given interval.

```
x = symbols('x', real=True)
f = x + x**2
interval = Interval(2, 3, left_open=True)
infimum = minimum(f, x, interval)
supremum = maximum(f, x, interval)
# Find points in the interval where each value is attained.
inf_points = solveset(f - infimum, x, domain=interval)
sup_points = solveset(f - supremum, x, domain=interval)
print("Infimum:", infimum)
print("Supremum:", supremum)
print("Infimum attained at:", inf_points)
print("Supremum attained at:", sup_points)
```

```
Infimum: 6
Supremum: 12
Infimum attained at: EmptySet
Supremum attained at: {3}
```

12. Write a SymPy statement using `Sum()` to represent $\sum_{r=1}^n r^2$ and print the same.

```
r, n = symbols('r n', integer=True, positive=True)
S = Sum(r**2, (r, 1, n))
print(S) # Use "pprint(S)" for symbolic output
```

```
Sum(r**2, (r, 1, n))
```

13. Use `Sum()` and `.doit()` to find the sum of cubes of the first n Natural numbers.

```
r, n = symbols('r n', integer=True, positive=True)
S = Sum(r**3, (r, 1, n))
print(factor(S.doit())) # Use "pprint(S)" for symbolic output
```

```
n**2*(n + 1)**2/4
```

14. Let $f(x) = x^3$. Use `f.subs()` to evaluate $f\left(\frac{r-1}{n}\right)$ and $f\left(\frac{r}{n}\right)$.

```
x = symbols('x', real=True)
f = x**3
print(f.subs(x, (r - 1)/n))
print(f.subs(x, r/n)) # Use "pprint(S)" for symbolic output
```

```
(r - 1)**3/n**3
r**3/n**3
```

15. Let `f = Sum(x**r, (r, 1, 4))`. What is the result of `f.subs(x, 2).doit()`?

```
x, r = symbols('x r')
f = Sum(x**r, (r, 1, 4))
print(f)      # Use "pprint(S)" for symbolic output
print(f.subs(x, 2))
print(f.subs(x, 2).doit())
```

```
Sum(x**r, (r, 1, 4))
Sum(2**r, (r, 1, 4))
30
```

16. Write a Python program to divide $[2, 5]$ into n equal sub-intervals. Find their common length and write the r th sub-interval.

```
n, r = symbols('n r', integer=True, positive=True)
a = 2
b = 5
length = (b - a)/n
left = a + (r - 1)*length
right = a + r*length
print("Common length:")
print(length)
print("The rth sub-interval is:")
print([left, right]) # Use "pprint(S)" for symbolic output
```

```
Common length:
3/n
The rth sub-interval is:
[2 + 3*(r - 1)/n, 2 + 3*r/n]
```

17. Write a Python program to divide $[1, 3]$ into eight equal sub-intervals. Find the fifth sub-interval.

```
a = S(1)
b = S(3)
length = (b - a)/8
left = a + 4*length
right = a + 5*length
print("The 5th sub-interval is:", pretty([left, right]))
```

```
The 5th sub-interval is: [2, 9/4]
```

Use `limit()` to evaluate

18. Evaluate $\lim_{x \rightarrow 0} \frac{\sin x}{x}$.

```
x = symbols('x')
print(limit(sin(x)/x, x, 0))
```

```
1
```

19. Evaluate $\lim_{x \rightarrow 0} \frac{e^x - 1}{x}$.

```
x = symbols('x')
print(limit((exp(x) - 1)/x, x, 0))
```

1

20. Evaluate $\lim_{n \rightarrow \infty} \left(1 + \frac{1}{n}\right)^n$.

```
n = symbols('n')
print(limit((1 + 1/n)**n, n, oo))
```

E

21. Evaluate $\lim_{x \rightarrow 0} \frac{1 - \cos x}{x^2}$.

```
x = symbols('x')
print(limit((1 - cos(x))/x**2, x, 0))
```

1/2

22. Evaluate $\lim_{x \rightarrow 0} \frac{\log(1+x)}{x}$.

```
x = symbols('x')
print(limit(log(1 + x)/x, x, 0))
```

1

23. Evaluate $\lim_{x \rightarrow \infty} \frac{3x^2 + 2x - 1}{2x^2 - 5x + 4}$.

```
x = symbols('x')
print(limit((3*x**2 + 2*x - 1)/(2*x**2 - 5*x + 4), x, oo))
```

3/2

Use **simplify()** to simplify

24. Simplify $\frac{x^2 - 1}{x - 1}$, where $x \neq 1$.

```
x = symbols('x')
print(simplify((x**2 - 1)/(x - 1)))
```

x + 1

25. Simplify $\sin^2 x + \cos^2 x$.

```
x = symbols('x')
print(simplify(sin(x)**2 + cos(x)**2))
```

1

26. Simplify $\frac{1}{x} + \frac{1}{x+1}$, where $x \neq 0, -1$.

```
x = symbols('x')
pprint(simplify(1/x + 1/(x + 1)))
```

$$\frac{2x + 1}{x(x + 1)}$$

Use `integrate()` to evaluate the following integrals.

27. Evaluate $\int (3x^2 + 2x - 5) dx$.

```
x = symbols('x', real=True)
print("The integral value is:", integrate(3*x**2 + 2*x - 5, x))
```

The integral value is: $x^3 + x^2 - 5x$

28. Evaluate $\int e^{2x} dx$.

```
x = symbols('x', real=True)
print("The integral value is:", integrate(exp(2*x), x))
```

The integral value is: $\exp(2x)/2$

29. Evaluate $\int x \cos x dx$.

```
x = symbols('x', real=True)
print("The integral value is:", integrate(x*cos(x), x))
```

The integral value is: $x \sin(x) + \cos(x)$

30. Evaluate $\int \frac{1}{1+x^2} dx$.

```
x = symbols('x', real=True)
print("The integral value is:", integrate(1/(1 + x**2), x))
```

The integral value is: $\text{atan}(x)$

31. Evaluate $\int \frac{2x}{x^2+1} dx$.

```
x = symbols('x', real=True)
print("The integral value is:", integrate(2*x/(x**2 + 1), x))
```

The integral value is: $\log(x^2 + 1)$

32. Evaluate $\int_0^1 (x^2 + x) dx$.

```
x = symbols('x', real=True)
print("The integral value is:", integrate(x**2 + x, (x, 0, 1)))
```

The integral value is: 5/6

33. Evaluate $\int_0^\pi \sin x dx$.

```
x = symbols('x', real=True)
print("The integral value is:", integrate(sin(x), (x, 0, pi)))
```

The integral value is: 2

34. Evaluate $\int_0^1 x e^x dx$.

```
x = symbols('x', real=True)
print("The integral value is:", integrate(x*exp(x), (x, 0, 1)))
```

The integral value is: 1

35. Evaluate $\int_0^1 \frac{1}{1+x^2} dx$.

```
x = symbols('x', real=True)
print("The integral value is:", integrate(1/(1 + x**2), (x, 0, 1)))
```

The integral value is: pi/4

36. Evaluate $\int_{-1}^1 x^3 dx$.

```
x = symbols('x', real=True)
print("The integral value is:", integrate(x**3, (x, -1, 1)))
```

The integral value is: 0

37. Evaluate $\int_0^\infty e^{-x} dx$.

```
x = symbols('x', real=True)
print("The integral value is:", integrate(exp(-x), (x, 0, oo)))
```

The integral value is: 1

38. Evaluate $\int_1^\infty \frac{1}{x^2} dx$.

```
x = symbols('x', real=True)
print("The integral value is:", integrate(1/x**2, (x, 1, oo)))
```

The integral value is: 1

39. Evaluate $\int_{-\infty}^0 e^x dx$.

```
x = symbols('x', real=True)
print("The integral value is:", integrate(exp(x), (x, -oo, 0)))
```

The integral value is: 1

40. Evaluate $\int_{-\infty}^{\infty} e^{-x^2} dx$.

```
x = symbols('x', real=True)
print("The integral value is:", integrate(exp(-x**2), (x, -oo, oo)))
```

The integral value is: sqrt(pi)

41. Evaluate $\int_{-\infty}^{\infty} \frac{1}{1+x^2} dx$.

```
x = symbols('x', real=True)
print("The integral value is:", integrate(1/(1 + x**2), (x, -oo, \
↪oo)))
```

The integral value is: pi

Evaluate the left-hand limit (LHL) and right-hand limit (RHL) of each function at the indicated point.

42. Find the LHL and RHL of $f(x) = \frac{|x|}{x}$ at $x = 0$.

```
x = symbols('x', real=True)
f = Abs(x)/x; point = 0
lhl = limit(f, x, point, dir='-')
rhl = limit(f, x, point, dir='+')
print("LHL =", lhl, "and RHL =", rhl)
```

LHL = -1 and RHL = 1

43. Find the LHL and RHL of $f(x) = \frac{1}{x}$ at $x = 0$.

```
x = symbols('x', real=True)
f = 1/x; point = 0
lhl = limit(f, x, point, dir='-')
rhl = limit(f, x, point, dir='+')
print("LHL =", lhl, "and RHL =", rhl)
```

LHL = $-\infty$ and RHL = ∞

44. Find the LHL and RHL of $f(x) = \frac{x+1}{x-2}$ at $x = 2$.

```
x = symbols('x', real=True)
f = (x + 1)/(x - 2); point = 2
lhl = limit(f, x, point, dir='-')
rhl = limit(f, x, point, dir='+')
print("LHL =", lhl, "and RHL =", rhl)
```

LHL = $-\infty$ and RHL = ∞

45. Find the LHL and RHL of $f(x) = e^{1/x}$ at $x = 0$.

```
x = symbols('x', real=True)
f = exp(1/x); point = 0
lhl = limit(f, x, point, dir='-')
rhl = limit(f, x, point, dir='+')
print("LHL =", lhl, "and RHL =", rhl)
```

LHL = 0 and RHL = ∞

46. Find the LHL and RHL of $f(x) = \tan^{-1}\left(\frac{1}{x}\right)$ at $x = 0$.

```
x = symbols('x', real=True)
f = atan(1/x); point = 0
lhl = limit(f, x, point, dir='-')
rhl = limit(f, x, point, dir='+')
print("LHL =", lhl, "and RHL =", rhl)
```

LHL = $-\pi/2$ and RHL = $\pi/2$

47. Find the LHL and RHL of $f(x) = [x]$ at $x = 2$, where $[x]$ denotes the greatest integer less than or equal to x .

```
x = symbols('x', real=True)
f = floor(x); point = 2
lhl = limit(f, x, point, dir='-')
rhl = limit(f, x, point, dir='+')
print("LHL =", lhl, "and RHL =", rhl)
```

LHL = 1 and RHL = 2

48. Find the LHL and RHL of $f(x) = \tan x$ at $x = \frac{\pi}{2}$.

```
x = symbols('x', real=True)
f = tan(x); point = pi/2
lhl = limit(f, x, point, dir='-')
rhl = limit(f, x, point, dir='+')
```

```
print("LHL =", lhl, "and RHL =", rhl)
```

LHL = ∞ and RHL = $-\infty$

49. Find the LHL and RHL of $f(x) = \begin{cases} x+2, & x < 1, \\ x^2, & x \geq 1, \end{cases}$ at $x = 1$.

```
x = symbols('x', real=True)
cases = [(x + 2, x**2, 1)]
for (left, right, point) in cases:
    lhl = limit(left, x, point, dir='-')
    rhl = limit(right, x, point, dir='+')
    print("LHL =", lhl, "and RHL =", rhl)
```

LHL = 3 and RHL = 1

50. Find the LHL and RHL of $f(x) = \begin{cases} \cos x, & x < 0, \\ \sin x, & x \geq 0, \end{cases}$ at $x = 0$.

```
x = symbols('x', real=True)
cases = [(cos(x), sin(x), 0)]
for (left, right, point) in cases:
    lhl = limit(left, x, point, dir='-')
    rhl = limit(right, x, point, dir='+')
    print("LHL =", lhl, "and RHL =", rhl)
```

LHL = 1 and RHL = 0

51. Find the LHL and RHL of $f(x) = \begin{cases} x^2 + 1, & x < 2, \\ 3x + 1, & x \geq 2, \end{cases}$ at $x = 2$.

```
x = symbols('x', real=True)
cases = [(x**2 + 1, 3*x + 1, 2)]
for (left, right, point) in cases:
    lhl = limit(left, x, point, dir='-')
    rhl = limit(right, x, point, dir='+')
    print("LHL =", lhl, "and RHL =", rhl)
```

LHL = 5 and RHL = 7

Use SymPy's `solve()` to solve the following equations or systems of equations.

52. Solve for x : $5x - 7 = 3x + 9$.

```
x = symbols('x', real=True)
print("Solution is x =", solve(Eq(5*x - 7, 3*x + 9), x))
```

Solution is $x = [8]$

53. Solve for x : $x^2 - 5x + 6 = 0$.

```
x = symbols('x', real=True)
print("Solution is x =", solve(x**2 - 5*x + 6, x))
```

Solution is $x = [2, 3]$

54. Solve for x : $x^3 - 6x^2 + 11x - 6 = 0$.

```
x = symbols('x', real=True)
print("Solution is x =", solve(x**3 - 6*x**2 + 11*x - 6, x))
```

Solution is $x = [1, 2, 3]$

55. Solve for x , where $x \neq 1, -2$: $\frac{1}{x-1} + \frac{2}{x+2} = 1$.

```
x = symbols('x', real=True)
print("Solution is x =", solve(1/(x - 1) + 2/(x + 2) - 1, x))
```

Solution is $x = [1 - \sqrt{3}, 1 + \sqrt{3}]$

56. Find all real solutions $\odot$: $\sqrt{x+2} = x$.

```
x = symbols('x', real=True)
print("Solution is x =", solve(sqrt(x + 2) - x, x))
```

Solution is $x = [2]$

57. Find all real solutions: $e^{2x} - 3e^x + 2 = 0$.

```
x = symbols('x', real=True)
print("Solution is x =", solve(exp(2*x) - 3*exp(x) + 2, x))
```

Solution is $x = [0, \log(2)]$

58. Find all real solutions: $\log x + \log(x-1) = \log 6$, where $x > 1$.

```
x = symbols('x', real=True)
print("Solution is x =", solve(log(x) + log(x - 1) - log(6), x))
```

Solution is $x = [3]$

59. Solve for x and y : $\begin{cases} 2x + 3y = 7, \\ 3x - 2y = 4. \end{cases}$

```
x,y = symbols('x y', real=True)
print("Solution is (x, y) =", solve([2*x + 3*y - 7, 3*x - 2*y - 4], (x, y), dict=True))
```

Solution is $(x, y) = [{x: 2, y: 1}]$

60. Solve for x and y : $\begin{cases} x^2 + y^2 = 25, \\ x + y = 7. \end{cases}$

```
x,y = symbols('x y', real=True)
print("Solution is (x, y) =", solve([x**2 + y**2 - 25, x + y - 7],
    ↪(x, y), dict=True))
```

Solution is $(x, y) = [\{x: 3, y: 4\}, \{x: 4, y: 3\}]$

61. Find all solutions of fourth root of unity: $x^4 - 1 = 0$.

```
z = symbols('z') # Allows complex solutions
print("Solution is (x, y) =", solve(z**4 - 1, z))
```

Solution is $(x, y) = [-1, 1, -I, I]$

Evaluate the expression using SymPy and use `.is_finite` to determine whether the result is finite.

62. Determine whether the value of the expression is finite: $\sqrt{2} + \frac{3}{7}$.

```
expr = sqrt(2) + S(3)/7
print("Result is:", expr, " and is finite:", expr.is_finite)
```

Result is: $3/7 + \sqrt{2}$ and is finite: True

63. Determine whether the value of the expression is finite: $3 + 4i$, where $i^2 = -1$.

```
expr = 3 + 4*I
print("Result is:", expr, " and is finite:", expr.is_finite)
```

Result is: $3 + 4*I$ and is finite: True

64. Determine whether the value of the expression is finite: $\lim_{x \rightarrow \infty} \frac{2x^2 + 1}{x^2 + 3}$.

```
expr = limit((2*x**2 + 1)/(x**2 + 3), x, oo)
print("Result is:", expr, " and is finite:", expr.is_finite)
```

Result is: 2 and is finite: True

65. Determine whether the value of the expression is finite: $\sum_{n=1}^{\infty} \frac{1}{n}$.

```
expr = Sum(1/n, (n, 1, oo)).doit()
print("Result is:", expr, " and is finite:", expr.is_finite)
```

Result is: oo and is finite: False

66. Determine whether the value of the expression is finite: $\int_0^{\infty} e^{-x} dx$.

```
expr = integrate(exp(-x), (x, 0, oo))
print("Result is:", expr, " and is finite:", expr.is_finite)
```

Result is: 1 and is finite: True

Evaluate the following using SymPy and identify whether the result is ∞ (positive infinity), $-\infty$ (negative infinity), ∞j (complex infinity), or nan (not a number).

67. Evaluate $\lim_{x \rightarrow 0^+} \frac{1}{x}$.

```
x = symbols('x')
print("Result is:", limit(1/x, x, 0, dir='+'))
```

Result is: ∞

68. Evaluate $\lim_{x \rightarrow 0^-} \frac{1}{x}$.

```
x = symbols('x')
print("Result is:", limit(1/x, x, 0, dir='-'))
```

Result is: $-\infty$

69. Evaluate $\lim_{x \rightarrow \infty} (x^3 - 2x)$.

```
x = symbols('x')
print("Result is:", limit(x**3 - 2*x, x, oo))
```

Result is: ∞

70. Evaluate $\lim_{x \rightarrow 0^+} \log x$.

```
x = symbols('x')
print("Result is:", limit(log(x), x, 0, dir='+'))
```

Result is: $-\infty$

71. Evaluate $\sum_{n=1}^{\infty} \frac{1}{n}$.

```
n = symbols('n')
print("Result is:", Sum(1/n, (n, 1, oo)).doit())
```

Result is: ∞

72. Evaluate $\int_1^{\infty} -\frac{1}{x} dx$.

```
x = symbols('x')
print("Result is:", integrate(-1/x, (x, 1, oo)))
```

Result is: -oo

73. Substitute $x = 0$ into $f(x) = \frac{1}{x}$ using `f.subs()`.

```
x = symbols('x', real=True)
f = 1/x
print("Result is:", f.subs(x, 0))
```

Result is: zoo

74. Evaluate the SymPy expression `S(5)/0`.

```
print("Result is:", S(5)/0)
```

Result is: zoo

75. Evaluate the SymPy expression `S(0)/0`.

```
print("Result is:", S(0)/0)
```

Result is: nan

76. Evaluate the SymPy expression `oo - oo`.

```
print("Result is:", oo - oo)
```

Result is: nan

Use SymPy's `diff()` to evaluate the following derivatives.

77. Find $\frac{dy}{dx}$ if $y = 4x^5 - 3x^3 + 2x - 7$.

```
x = symbols('x')
y = 4*x**5 - 3*x**3 + 2*x - 7
print("Derivative of y is:", diff(y, x))
```

Derivative of y is: $20x^4 - 9x^2 + 2$

78. Find $\frac{dy}{dx}$ if $y = x^2e^x$.

```
x = symbols('x')
y = x**2*exp(x)
print("Derivative of y is:", diff(y, x))
```

Derivative of y is: $x^2e^x + 2xe^x$

79. Find $\frac{dy}{dx}$ if $y = \frac{x^2 + 1}{x^2 - 1}$.

```
x = symbols('x')
y = (x**2 + 1)/(x**2 - 1)
print("Derivative of y is:", simplify(diff(y, x)))
```

Derivative of y is: $-4x/(x^2 - 1)^2$

80. Find $\frac{dy}{dx}$ if $y = \sin(x^2 + 1)$.

```
x = symbols('x')
y = sin(x**2 + 1)
print("Derivative of y is:", diff(y, x))
```

Derivative of y is: $2x \cos(x^2 + 1)$

81. Find $\frac{dy}{dx}$ if $y = \log(1 + x^2)$.

```
x = symbols('x')
y = log(1 + x**2)
print("Derivative of y is:", diff(y, x))
```

Derivative of y is: $2x/(x^2 + 1)$

82. Find $\frac{dy}{dx}$ if $y = \tan^{-1}(2x)$.

```
x = symbols('x')
y = atan(2*x)
print("Derivative of y is:", diff(y, x))
```

Derivative of y is: $2/(4x^2 + 1)$

83. Find $\frac{d^2y}{dx^2}$ if $y = e^x \sin x$.

```
x = symbols('x')
y = exp(x)*sin(x)
print("Second derivative of y is:", diff(y, x, 2))
```

Derivative of y is: $2 \exp(x) \cos(x)$

84. Find $\frac{d^4y}{dx^4}$ if $y = x^6 - 2x^4 + 3x^2$.

```
x = symbols('x')
y = x**6 - 2*x**4 + 3*x**2
print("Forth derivative of y is:", simplify(diff(y, x, 4)))
```

Forth derivative of y is: $360x^2 - 48$

85. Find $\frac{\partial f}{\partial x}$ and $\frac{\partial f}{\partial y}$ if $f(x, y) = x^3y^2 + e^{xy}$.

```
x, y = symbols('x y')
f = x**3*y**2 + exp(x*y)
print("Partial derivative of f w.r.t x is:", diff(f, x))
```

```
print("Partial derivative of f w.r.t y is:", diff(f, y))
```

Partial derivative of f w.r.t x is: $3x^2y^2 + y\exp(xy)$

Partial derivative of f w.r.t y is: $2x^3y + x\exp(xy)$

86. Find $\frac{\partial^2 f}{\partial y \partial x}$ and $\frac{\partial^6 f}{\partial x^2 \partial y^4}$ if $f(x, y) = x^2 \sin y + y^2 \cos x$.

```
x, y = symbols('x y')
g = x**2*sin(y) + y**2*cos(x)
print("Second order partial derivative of g is:", diff(g, y, x))
print("Sixth order partial derivative of g is:", diff(g, x, 2, y, 4))
```

Second order partial derivative of g is: $2(x\cos(y) - y\sin(x))$

Sixth order partial derivative of g is: $2\sin(y)$

For each question, evaluate the expression using SymPy, then use `.is_positive` and `.is_negative` to determine its sign.

87. Determine the sign of $\frac{3}{4} - \frac{5}{6}$.

```
expr = S(3)/4 - S(5)/6
print("Result =", expr, "Positive =", expr.is_positive, "and_
Negative =", expr.is_negative)
```

Result = $-1/12$ Positive = False and Negative = True

88. Determine the sign of $\sqrt{5} - 2$.

```
expr = sqrt(5) - 2
print("Result =", expr, "Positive =", expr.is_positive, "and_
Negative =", expr.is_negative)
```

Result = $-2 + \sqrt{5}$ Positive = True and Negative = False

89. Determine the sign of $\log\left(\frac{1}{2}\right)$.

```
expr = log(S(1)/2)
print("Result =", expr, "Positive =", expr.is_positive, "and_
Negative =", expr.is_negative)
```

Result = $-\log(2)$ Positive = False and Negative = True

90. Determine the sign of $\cos\left(\frac{2\pi}{3}\right)$.

```
expr = cos(2*pi/3)
print("Result =", expr, "Positive =", expr.is_positive, "and_
Negative =", expr.is_negative)
```

Result = -1/2 Positive = False and Negative = True

91. Determine the sign of $e^2 - 1$.

```
expr = exp(2) - 1
print("Result =", expr, "Positive =", expr.is_positive, "and_
↳Negative =", expr.is_negative)
```

Result = -1 + exp(2) Positive = True and Negative = False

92. Determine the sign of $\sin^2\left(\frac{\pi}{4}\right) + \cos^2\left(\frac{\pi}{4}\right) - 1$.

```
expr = sin(pi/4)**2 + cos(pi/4)**2 - 1
print("Result =", expr, "Positive =", expr.is_positive, "and_
↳Negative =", expr.is_negative)
```

Result = 0 Positive = False and Negative = False

93. Find the sign of $f'(1)$, where $f(x) = x^3 - 6x$.

```
x = symbols('x')
f = x**3 - 6*x
expr = diff(f, x).subs(x, 1)
print("Result =", expr, "Positive =", expr.is_positive, "and_
↳Negative =", expr.is_negative)
```

Result = -3 Positive = False and Negative = True

94. Find the sign of $\int_0^1 (1 - x^2) dx$.

```
x = symbols('x')
expr = integrate(1 - x**2, (x, 0, 1))
print("Result =", expr, "Positive =", expr.is_positive, "and_
↳Negative =", expr.is_negative)
```

Result = 2/3 Positive = True and Negative = False

95. Find the sign of $\lim_{x \rightarrow 0} \frac{\cos x - 1}{x^2}$.

```
x = symbols('x')
expr = limit((cos(x) - 1)/x**2, x, 0)
print("Result =", expr, "Positive =", expr.is_positive, "and_
↳Negative =", expr.is_negative)
```

Result = -1/2 Positive = False and Negative = True

96. For a real symbol x , determine the sign of $x^2 + 1$.

```
x = symbols('x', real=True)
expr = x**2 + 1
```

```
print("Result =", expr, "Positive =", expr.is_positive, "and_
      ↪Negative =", expr.is_negative)
```

Result = x**2 + 1 Positive = True and Negative = False

97. Use `gamma()` to evaluate $\Gamma(6)$.

```
print("Gamma function evaluated value:", gamma(6))
```

Gamma function evaluated value: 120

98. Use `gamma()` to evaluate $\Gamma\left(\frac{7}{2}\right)$.

```
print("Gamma function evaluated value:", gamma(S(7)/2))
```

Gamma function evaluated value: 15*sqrt(pi)/8

99. Use `beta()` to evaluate $B(3, 4)$.

```
print("Beta function evaluated value:", beta(3, 4))
```

Beta function evaluated value: 1/60

100. Use `beta()`, `gamma()` and `.evalf()` to verify $B\left(\frac{1}{2}, \frac{3}{2}\right) = \frac{\Gamma\left(\frac{1}{2}\right)\Gamma\left(\frac{3}{2}\right)}{\Gamma(2)}$.

```
a = S(1)/2
b = S(3)/2
lhs = beta(a, b)
rhs = gamma(a)*gamma(b)/gamma(a + b)
print("LHS =", lhs)
print("RHS =", rhs)
print("Identity verified:", lhs.evalf() - rhs.evalf() == 0)
```

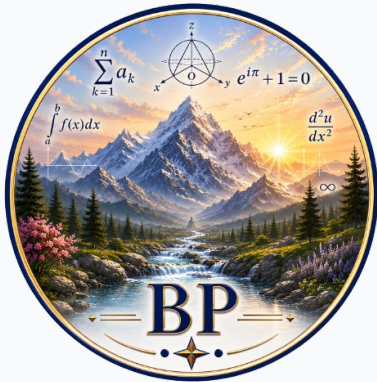
LHS = 4/(3*catalan(1/2))

RHS = pi/2

Identity verified: True

Bibliography

- [1] N. P. Bali, *Real Analysis for B.Sc. Students*, 1st ed., Firewall Media (An Imprint of Laxmi Publications Pvt. Ltd.), New Delhi, 2009, reprinted 2016. ISBN 978-81-318-0503-9.
- [2] S. K. Mukherjee, *Topics in Real Analysis*, 3rd ed., Academic Publishers, Kolkata. ISBN: 978-93-83420-48-3.
- [3] R. K. Yadav, *Real Analysis*, Laxmi Publications Pvt. Ltd., New Delhi, 2022. ISBN: 978-93-95609-35-7.
- [4] Hugo D. Junghenn, *Symbolic Mathematics with Python*, Synthesis Lectures on Mathematics & Statistics, 1st ed., Springer, Cham, 2025. ISBN: 978-3-031-90521-6. doi: <https://doi.org/10.1007/978-3-031-90522-3>
- [5] A. Saha, *Doing Math with Python: Use Programming to Explore Algebra, Statistics, Calculus, and More!*, No Starch Press, San Francisco, 2015. ISBN: 978-1-59327-640-9.
- [6] J. M. Stewart, *Python for Scientists*, 2nd ed., Cambridge University Press, Cambridge, 2017. doi: <https://doi.org/10.1017/9781108120241>. ISBN: 978-1-108-12024-1.
- [7] A. Meurer, C. P. Smith, M. Paprocki, O. Čertík, S. B. Kirpichev, M. Rocklin, A. Kumar, S. Ivanov, J. K. Moore, S. Singh, T. Rathnayake, S. Vig, B. E. Granger, R. P. Muller, F. Bonazzi, H. Gupta, S. Vats, F. Johansson, F. Pedregosa, M. J. Curry, A. R. Terrel, Š. Roučka, A. Saboo, I. Fernando, S. Kulal, R. Cimrman, and A. Scopatz, “SymPy: Symbolic Computing in Python”, *PeerJ Computer Science*, vol. 3, p. e103, 2017. doi: <https://doi.org/10.7717/peerj-cs.103>.
- [8] Various OpenAIs, *Large Language Models*, 2026.
- [9] SymPy Development Team, *SymPy Documentation*, <https://docs.sympy.org/>, accessed August 2, 2026.



PRACTICALS ON REAL ANALYSIS – II WITH PYTHON

A Laboratory Manual for Undergraduate Mathematics.

EXPLORE ■ COMPUTE ■ UNDERSTAND

From Concepts to Computations, from Theory to Practice.

ABOUT THIS MANUAL

This laboratory manual integrates the fundamental ideas of Real Analysis–II [C5MAT1P1–Karnatak University, Dharwad] with computational methods implemented in Python.

It aims to help undergraduate students explore mathematical examples, carry out symbolic experiments and develop computational intuition while strengthening their mathematical reasoning. By bringing theory and computation together, it seeks to make learning Real Analysis a more active, insightful and rewarding experience.

KEY FEATURES

- Structured Python Experiments
- 100 Quick Python Exercises
- Exact Outputs
- Clear Concept Introduction
- Different Practice Sets
- Revised NEP-aligned Curriculum

$$L(P, f) \int_a^b f(x) dx U(P, f)$$

Dr. B Parvathalu

M.Sc., M.Tech., Ph.D.

Karnatak University's
Karnatak Arts/Science College, Dharwad
Karnataka, India

C5MAT1P1 (2026–27)

Educational Use Only

ISBN: 978-93-344-8833-3

